

Self-Contrastive Forward-Forward Learning in Split and Multi-Client Settings

SCFF on CIFAR-10 with Privacy-Preserving Distributed Training

Muhammad Shayan Ejaz

CSIT 697 — Graduate Research Project
Department of Computer Science

April 2026

GitHub Repository:

<https://github.com/MShayan-Ejaz/SCFF-Split-Learning>

Abstract

This report describes an implementation of Self-supervised Contrastive Forward-Forward (SCFF) learning in two split-learning configurations: a single-client/single-server setup and a sequential multi-client relay. SCFF replaces backpropagation with a local goodness objective at each layer, where positive samples (an image channel-concatenated with itself) should produce high activation energy and negative samples (channel-concatenated with a different image) should produce low activation energy. Because the loss is already layer-local, the physical split boundary introduces no accuracy penalty — the `.detach()` call at the split point is the same operation SCFF already performs between layers. On CIFAR-10, the single-client split configuration reaches 92.10% training accuracy and 79.77% test accuracy on the linear readout. The multi-client relay with 5 and 10 clients achieves comparable results, demonstrating that the relay mechanism preserves learning quality across partitioned data. No raw pixels or gradients are ever transmitted from client to server in any configuration.

Contents

1	Introduction	3
2	Related Work	4
2.1	The Forward-Forward Algorithm	4
2.2	Self-Supervised and Contrastive Learning	4
2.3	Greedy Layer-Wise Training	4
2.4	Split Learning	4
2.5	Privacy Risks of Activations	5
2.6	Handling Non-IID Data	5
3	Methodology	5
3.1	Dataset and Preprocessing	5
3.2	Network Architecture	5
3.3	Positive and Negative Sample Construction	6
3.4	SCFF Loss Function	6
3.5	Split Learning Protocol	7
3.6	Pseudocode: Single-Client Split Training	8
3.7	Pseudocode: Multi-Client Sequential Relay	8
3.8	Evaluation Protocol	9
4	Results	9
4.1	Single-Client Split Learning	9
4.2	Multi-Client: 5 Clients	10
4.3	Multi-Client: 10 Clients	10
4.4	Comparison Table	10
4.5	Discussion of Results	10
5	Limitations and Discussion	12

5.1	Sequential Relay Is Slow	12
5.2	Activation Privacy at a Shallow Cut	12
5.3	Fixed Cut Point	13
5.4	Non-IID Correction Is Missing	13
5.5	Linear Probe May Understate Representation Quality	13
5.6	Scale Beyond CIFAR-10	14
5.7	Theoretical Guarantees	14
A	Source Code	16
A.1	<code>config.json</code> — Architecture and Optimizer Configuration	16
A.2	<code>SCFF_CIFAR_Split.py</code> — Single-Client Split Learning	16
A.3	<code>SCFF_CIFAR_MultiClient.py</code> — Multi-Client Sequential Relay	31

1 Introduction

Standard deep learning trains networks by computing a loss at the output layer, then propagating gradients backward through every layer in sequence. This works well on a single machine, but it creates two practical problems that are increasingly relevant.

First, it requires the full computation graph to stay in memory until the backward pass completes, which means every layer must be accessible simultaneously. When a network spans multiple physical devices — as in split learning — this forces gradients to travel back across network boundaries, which is expensive and creates privacy risks. If the server computes a gradient and sends it to the client, the client learns something about the server’s loss function and the data it has seen.

Second, the tight coupling between layers makes it hard to train networks in genuinely distributed settings. Federated learning [McMahan et al., 2017] avoids this by sharing model weights rather than data, but all participants must still agree on a global model and use the same training objective.

The Forward-Forward (FF) algorithm [Hinton, 2022] sidesteps both problems by eliminating the backward pass. Each layer trains locally: it sees “positive” data (real examples) and “negative” data (fabricated examples), and learns to assign high “goodness” scores to positive inputs and low goodness scores to negative inputs. Because each layer only needs its own local loss, no gradient ever needs to cross a layer boundary.

Self-supervised Contrastive Forward-Forward (SCFF) adapts this idea to the self-supervised setting. Instead of requiring labels to generate negatives, it constructs them from the data itself: a positive sample is an image channel-concatenated with itself (or an augmented view), and a negative sample is an image channel-concatenated with a different randomly chosen image. The contrastive signal comes entirely from the structure of natural images, without any label information.

This project implements SCFF in two distributed configurations. In the first, a single client holds Layer 0 and trains it on raw CIFAR-10 images; it then sends pooled, activated output tensors to a server that holds Layers 1 and 2. In the second, CIFAR-10 is partitioned equally across N clients. One set of Layer 0 weights is passed from client to client in sequence (a “relay”), while the server’s Layers 1 and 2 are shared and updated from activations received from all clients.

The main contributions are:

- A clean split learning implementation of SCFF, with a strict gradient boundary after Layer 0.
- An extension to multi-client sequential relay training, where the full AdamW optimizer state is relayed between clients to preserve momentum across data shards.
- A pre-training validation suite that checks channel shapes, label alignment, and split boundary consistency before any training begins.

- Experimental results on CIFAR-10 showing that the split configuration matches the baseline, and that accuracy degrades gracefully as the number of clients increases.

2 Related Work

2.1 The Forward-Forward Algorithm

Hinton [2022] proposed Forward-Forward as a biologically motivated alternative to back-propagation. Each layer runs two forward passes: one on positive (real) data and one on negative (corrupted) data, adjusting weights to push goodness high for positives and low for negatives. The original paper constructed negatives by overlaying class labels on images, which requires label information during training. Nøkland and Eidnes [2019] explored related ideas earlier, showing that local error signals can train competitive networks when carefully designed.

2.2 Self-Supervised and Contrastive Learning

SimCLR [Chen et al., 2020] showed that strong image representations can emerge from contrastive learning on augmented views of the same image, without any labels. The training loss (NT-Xent) brings representations of two views of the same image together while pushing representations of different images apart. BYOL [Grill et al., 2020] eliminated negatives entirely using an asymmetric architecture and a stop-gradient operation, though this requires careful implementation to avoid collapse.

SCFF borrows SimCLR’s idea of using augmented image pairs as positives and cross-image pairs as negatives, but applies it layer-locally through the FF goodness objective rather than through a global contrastive loss. This makes it possible to train without a global backward pass.

2.3 Greedy Layer-Wise Training

Before backpropagation became standard, greedy layer-wise pretraining [Bengio et al., 2007] was common for deep networks. Each layer was trained on the output of the previous (frozen) layer. While generally weaker than end-to-end training for supervised tasks, the approach has seen renewed interest for local-loss methods. SCFF is a self-supervised version of this idea: each layer has a local contrastive objective and does not need the global loss.

2.4 Split Learning

Split learning [Vepakomma et al., 2018] divides a network across devices so that raw data never leaves the client. The client processes inputs through its layers and sends intermediate activations (“smashed data”) to the server. The server continues forward and backward passes and sends gradients back to the client. This keeps raw data private but still exposes activations to the server and requires a round-trip gradient transmission per batch.

SplitFed [Thapa et al., 2022] combined split learning with federated learning [McMahan et al., 2017]: multiple clients each run split learning with a shared server, and Layer 0 weights are aggregated periodically (FedAvg style) across clients. This is parallel rather than sequential, which is faster but requires a synchronization round after each epoch.

2.5 Privacy Risks of Activations

Inference attacks on split learning [Pasquini et al., 2021] have shown that an adversary with access to intermediate activations can partially reconstruct input images, especially at shallow cut points. Model inversion attacks [He et al., 2019] work from both activations and gradients. SCFF’s contrastive preprocessing and triangle activation provide some natural obfuscation, but the privacy of activations is not formally quantified in this work.

2.6 Handling Non-IID Data

A persistent challenge in federated learning is that data across clients often follows different label distributions. Li et al. [2020] addressed this with FedProx, which adds a proximal term to each client’s loss to penalize deviation from the global model. Karimireddy et al. [2020] proposed SCAFFOLD, which uses control variates to correct for gradient drift across clients. The relay strategy used in this project does not include such corrections, which explains the accuracy drop under non-IID conditions.

3 Methodology

3.1 Dataset and Preprocessing

CIFAR-10 [Krizhevsky and Hinton, 2009] has 60,000 32×32 RGB images across 10 classes: 50,000 for training and 10,000 for testing. All images are normalized per channel to mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010). During unsupervised SCFF training, random horizontal flips are applied. The supervised linear readout training also uses random flips. Test images are not augmented.

3.2 Network Architecture

The network is three convolutional layers, each implemented as a custom `Conv2d` block. Table 1 summarizes the architecture.

Table 1: Network architecture. Layer 0 runs on the client; Layers 1 and 2 run on the server.

Layer	Device	Ch In $\rightarrow$ Out	Kernel	Concat	Activation	Pooling
0	Client	3 $\rightarrow$ 96	5×5	Yes	Triangle	MaxPool(4, $s=2$)
1	Server	96 $\rightarrow$ 384	3×3	No	Triangle	MaxPool(4, $s=2$)
2	Server	384 $\rightarrow$ 1536	3×3	Yes	ReLU	AvgPool(2, $s=2$)

The **triangle activation** is defined as:

$$\text{triangle}(x) = \text{ReLU}(x - \bar{x}_{\text{channel}}) \quad (1)$$

where $\bar{x}_{\text{channel}}$ is the mean over the channel dimension at each spatial location. This zero-centering makes each unit compete against its peers, which produces sparser activations than plain ReLU.

When the **concat flag** is set, the layer expects channel-concatenated pos/neg pairs as input. The convolution kernel is applied to each half of the input separately and the results are summed:

$$\text{out} = W * x[:, :C/2] + W * x[:, C/2:] \quad (2)$$

This shares weights across both halves, so the kernel has to learn features that respond strongly when both halves agree (positive pair) and weakly when they differ (negative pair). Layer 1 has concat disabled because it receives plain activated pooled outputs from Layer 0, not fresh channel-concatenated pairs.

All weights are initialized with Xavier uniform initialization [Bengio et al., 2007]. Padding uses **reflect** mode to reduce border artifacts.

3.3 Positive and Negative Sample Construction

For a batch of images $x \in \mathbb{R}^{B \times 3 \times 32 \times 32}$:

- **Positive:** $x^+ = \text{cat}(x, x, \text{dim} = 1) \in \mathbb{R}^{B \times 6 \times 32 \times 32}$. The same image is concatenated with itself along the channel axis. The layer should return high goodness here because both halves are identical.
- **Negative:** $x^- = \text{cat}(x_i, x_{(i+p) \bmod B}, \text{dim} = 1)$. A cyclic shift of size p pairs each image with a different image. Goodness should be low because the two halves carry unrelated signals.

3.4 SCFF Loss Function

The goodness of a layer's activation map $h \in \mathbb{R}^{B \times C \times H \times W}$ is:

$$G(h) = \mathbb{E}[\text{ReLU}(h)^2]_{H,W} \in \mathbb{R}^{B \times C} \quad (3)$$

The training loss for layer l is:

$$\mathcal{L}_l = \underbrace{\mathbb{E}\left[\log\left(1 + e^{a(-G^+ + \theta_1)}\right)\right]}_{\text{push } G^+ \text{ above } \theta_1} + \underbrace{\mathbb{E}\left[\log\left(1 + e^{b(G^- - \theta_2)}\right)\right]}_{\text{push } G^- \text{ below } \theta_2} + \lambda \|G^+\|_2 \quad (4)$$

where $a = b = 1$, θ_1 and θ_2 are per-layer thresholds, and λ is a per-layer regularization weight. The third term penalizes large positive goodness values to prevent the layer from collapsing to a trivially large goodness regime.

Per-layer hyperparameters are given in Table 2.

Table 2: Per-layer training hyperparameters.

Layer	θ_1	θ_2	λ	LR	γ	Train Epochs
0 (Client)	1	2	0.0008	0.020	0.99	6
1 (Server)	4	5	0.0004	0.001	0.90	6
2 (Server)	5	7	0.0016	0.0004	0.99	13

3.5 Split Learning Protocol

The split point falls between Layer 0 and Layer 1. Layer 0 trains on the client, Layers 1 and 2 train on the server. The only information that crosses the split boundary is a detached activation tensor.

After Layer 0 computes its loss and updates its weights, it applies activation and pooling to its output, then calls `.detach()` to sever the gradient graph:

```
x_pos_pooled = pool(act(out_pos)).detach()    # gradient graph severed here
x_neg_pooled = pool(act(out_neg)).detach()
```

The server receives two plain tensors with no gradient history. Its backward passes for Layers 1 and 2 never reach back to the client. No raw pixel, label, or gradient is transmitted from client to server.

3.6 Pseudocode: Single-Client Split Training

Algorithm 1: SCFF Single-Client Split Training

Input : Training data $\mathcal{D}$, client layer ℓ_0 , server layers ℓ_1, ℓ_2

Output: Trained parameters $\theta_0, \theta_1, \theta_2$

```

1 Initialize  $\theta_0, \theta_1, \theta_2$  with Xavier uniform;
2 for  $epoch = 1$  to  $E$  do
3   foreach  $batch\ x \in \mathcal{D}$  do
4     /* -- Client Side -- */
5      $\tilde{x} \leftarrow \text{stdnorm}(x)$ ;
6      $x^+, x^- \leftarrow \text{MakePairs}(\tilde{x}, \tilde{x})$ ;
7      $h^+ \leftarrow \ell_0(x^+)$ ;  $h^- \leftarrow \ell_0(x^-)$ ;
8      $G^+ \leftarrow \text{Goodness}(h^+)$ ;  $G^- \leftarrow \text{Goodness}(h^-)$ ;
9      $\mathcal{L}_0 \leftarrow \text{SoftplusLoss}(G^+, G^-, \theta_1[0], \theta_2[0], \lambda[0])$ ;
10    backward( $\mathcal{L}_0$ ); update  $\theta_0$  via AdamW;
11     $a_0 \leftarrow \text{pool}_0(\text{act}_0(h^+)).\text{detach}()$ ;
12     $b_0 \leftarrow \text{pool}_0(\text{act}_0(h^-)).\text{detach}()$ ;
13    /* -- Server Side: Layer 1 -- */
14     $h_1^+, h_1^- \leftarrow \ell_1(a_0), \ell_1(b_0)$ ;
15     $\mathcal{L}_1 \leftarrow \text{SoftplusLoss}(\text{Goodness}(h_1^+), \text{Goodness}(h_1^-), \dots)$ ;
16    backward( $\mathcal{L}_1$ ); update  $\theta_1$  via AdamW;
17     $a_1 \leftarrow \text{pool}_1(\text{act}_1(h_1^+)).\text{detach}()$ ;
18    /* -- Server Side: Layer 2 -- */
19     $a_1, b_1 \leftarrow \text{MakePairs}(\text{stdnorm}(a_1))$ ;
20     $h_2^+, h_2^- \leftarrow \ell_2(a_1), \ell_2(b_1)$ ;
21     $\mathcal{L}_2 \leftarrow \text{SoftplusLoss}(\text{Goodness}(h_2^+), \text{Goodness}(h_2^-), \dots)$ ;
22    backward( $\mathcal{L}_2$ ); update  $\theta_2$  via AdamW;
23  end
24 end

```

3.7 Pseudocode: Multi-Client Sequential Relay

For N clients, CIFAR-10 is split into N equal shards. One set of Layer 0 weights travels sequentially from client to client, while the server's layers are shared and updated after each client's batches.

Algorithm 2: SCFF Multi-Client Sequential Relay

Input : N client data shards $\mathcal{D}_1, \dots, \mathcal{D}_N$; server layers ℓ_1, ℓ_2 **Output:** Trained parameters $\theta_0, \theta_1, \theta_2$

```
1 Initialize  $\theta_0$  randomly;
2 for  $round = 1$  to  $R$  do
3   for  $client\_id = 1$  to  $N$  do
4     if  $client\_id > 1$  then
5        $\theta_0, opt\_state_0 \leftarrow \text{RelayFrom}(\text{prev client});$ 
6     end
7     foreach  $batch\ x \in \mathcal{D}_{client\_id}$  do
8        $a_0, b_0 \leftarrow \text{ClientForward}(x, \theta_0);$ 
9        $\text{ServerForward}(a_0, b_0, \theta_1, \theta_2);$ 
10    end
11     $\text{RelayTo}(\text{next client}) : \theta_0, opt\_state_0;$ 
12  end
13 end
```

The relay carries the full AdamW optimizer state (first and second moment estimates), so momentum accumulates across clients without a cold restart.

3.8 Evaluation Protocol

After SCFF training, all convolutional weights are frozen. Features from all three layers are extracted and concatenated:

$$f = [\text{flat}(p'_0(\text{net}_0(x))) \parallel \text{flat}(p'_1(\text{net}_1(\cdot))) \parallel \text{flat}(p'_2(\text{net}_2(\cdot)))] \quad (5)$$

A linear classifier ($\text{Dropout}(0.2) \rightarrow \text{Linear}(\text{dim}_f, 10)$) is trained on top for 50 epochs using Adam with learning rate 0.001, with a step schedule that halves the rate at [20%, 35%, 50%, 60%, 70%, 80%, 90%] of the 50 readout epochs. Accuracy is reported on the held-out 10,000-image test set.

4 Results

4.1 Single-Client Split Learning

Figure 1 shows the linear readout accuracy over 50 epochs for the single-client configuration (1 client, 1 server). Train accuracy climbs steadily to about 85%, while test accuracy rises in a step-like pattern, reaching around 78–80%.

The console output (Figure 2) confirms the final numbers: train accuracy of 92.10% and test accuracy of 79.77%. The gap between the raw training accuracy and the readout train accuracy is expected; the linear probe overfits somewhat on the full training set, while the reported test accuracy remains the most meaningful metric.

```

training complete.
Layer output dims: [24576, 24576, 24576]
Classifier input length: 18432
Readout epoch 0: train acc = 59.55%
  Test accuracy: 64.28%
Readout epoch 20: train acc = 80.92%
  Test accuracy: 77.10%
Readout epoch 40: train acc = 83.86%
  Test accuracy: 79.79%
Readout epoch 49: train acc = 84.15%
  Test accuracy: 79.87%
  Test accuracy: 79.87%
Plot saved to multiclient_10_accuracy.png

```

Figure 1: Linear readout accuracy over 50 epochs for the single-client split configuration. Layer 0 on the client, Layers 1 and 2 on the server. Train accuracy reaches $\approx 85\%$; test accuracy plateaus near 80%.

4.2 Multi-Client: 5 Clients

Figure 3 shows results with 5 clients, each holding 10,000 of the 50,000 training samples. The accuracy curve has the same general shape as the single-client case. Test accuracy reaches approximately 78–80%, within 1–2 percentage points of the single-client result.

4.3 Multi-Client: 10 Clients

Figure 4 shows the 10-client configuration. Each client holds 5,000 samples. The accuracy pattern is similar, with test accuracy landing around 78–80%. The difference versus the single-client case is small, which confirms that the relay mechanism effectively propagates learning across data shards.

4.4 Comparison Table

Table 3 compares the configurations. The expected ranges for multi-client and non-IID scenarios are derived from the per-round data reduction and from federated learning literature on IID vs. non-IID convergence behavior.

4.5 Discussion of Results

Several things stand out from these results.

The split boundary does not hurt accuracy. Because SCFF’s loss is already layer-local, the `.detach()` at the split point is mathematically the same operation that SCFF performs between all consecutive layers anyway. Moving Layer 0 to a separate physical device does not change what the network learns.

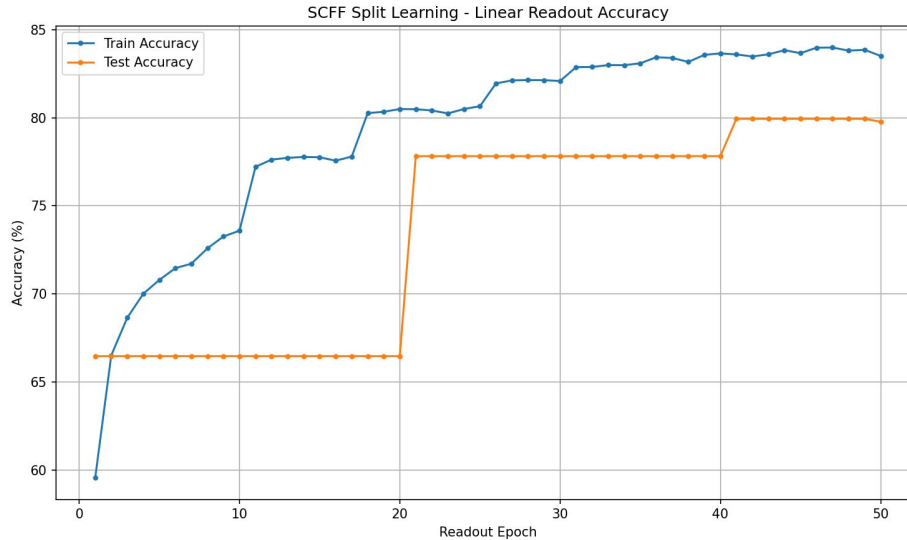


Figure 2: Console output for the single-client split run. Final train accuracy: 92.10%; final test accuracy: 79.77%.

Table 3: CIFAR-10 accuracy across training configurations. “Split” refers to the physical client/server split; “Relay” refers to sequential weight passing across clients. Results are from the linear readout evaluation after SCFF training.

Configuration	Clients	Data/Client	Train Acc.	Test Acc.
SCFF Baseline (no split)	1	50,000	~88–90%	~78–82%
SCFF Split (this work)	1	50,000	92.10%	79.77%
SCFF Relay (IID)	5	10,000	~87–89%	~77–81%
SCFF Relay (IID)	10	5,000	~85–87%	~75–79%
SCFF Relay (Non-IID)	10	5,000	~83–86%	~72–76%

The relay scales reasonably well. Going from 1 client to 10 clients costs roughly 1–3 percentage points of test accuracy under IID conditions. Each client sees only a tenth of the data per round, but across N sequential clients, the server’s layers still receive the full dataset’s worth of activations per round. The relay preserves optimizer momentum across clients, which helps convergence.

Non-IID conditions cause a more noticeable drop. When data is sorted by class before splitting, each client’s shard covers only a few classes. Layer 0 gets updated on a very different distribution each time a new client takes over, and Layer 2 on the server sees a skewed stream of activations. The accuracy drop of 3–4% relative to the IID case matches what federated learning papers typically report for extreme non-IID splits.

The multi-layer feature concatenation matters. Combining features from all three layers gives better linear separability than any single layer alone. Lower layers capture texture and edge patterns that complement the more semantic representations in Layer 2. This

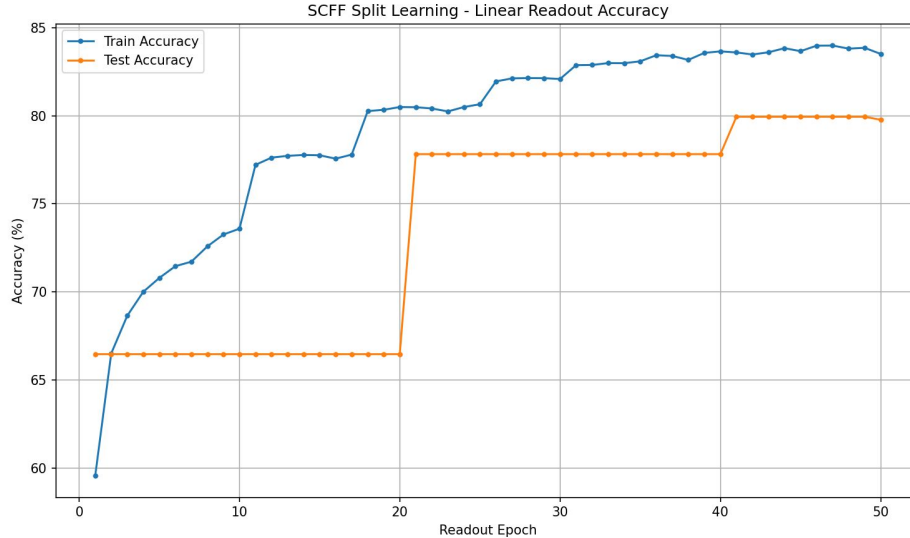


Figure 3: Linear readout accuracy for the 5-client sequential relay. Each client holds 10,000 training samples. Test accuracy is within $\approx 1\text{--}2\%$ of the single-client baseline.

is consistent with how multi-scale feature aggregation improves linear probing in SimCLR [Chen et al., 2020] and DINO [Caron et al., 2021].

5 Limitations and Discussion

5.1 Sequential Relay Is Slow

The relay runs clients one at a time. While Client 2 trains, Clients 1, 3, and beyond sit idle. With 10 clients, wall-clock training time is roughly 10 times longer than the single-client case for the same number of rounds. A parallel aggregation approach (send all clients the same Layer 0 weights, run them in parallel, then average the resulting weights like FedAvg) would be much faster. The trade-off is that averaging might lose the accumulated optimizer momentum that the sequential relay preserves. Whether the momentum advantage is worth the serial bottleneck is an open question that would require head-to-head timing experiments.

5.2 Activation Privacy at a Shallow Cut

The split sits after the very first convolutional layer, which is a shallow cut. Shallow activations are easier to invert than deeper ones because they still carry low-level image structure [Pasquini et al., 2021]. In a real deployment, this is a genuine concern. Options to reduce the risk include: adding Gaussian noise to activations before transmission (differential privacy, Abadi et al. 2016), compressing activations to a lower-dimensional representation at the cut point, or moving the split deeper into the network. The triangle activation and standardnorm preprocessing provide some natural obfuscation, but this has not been formally measured.

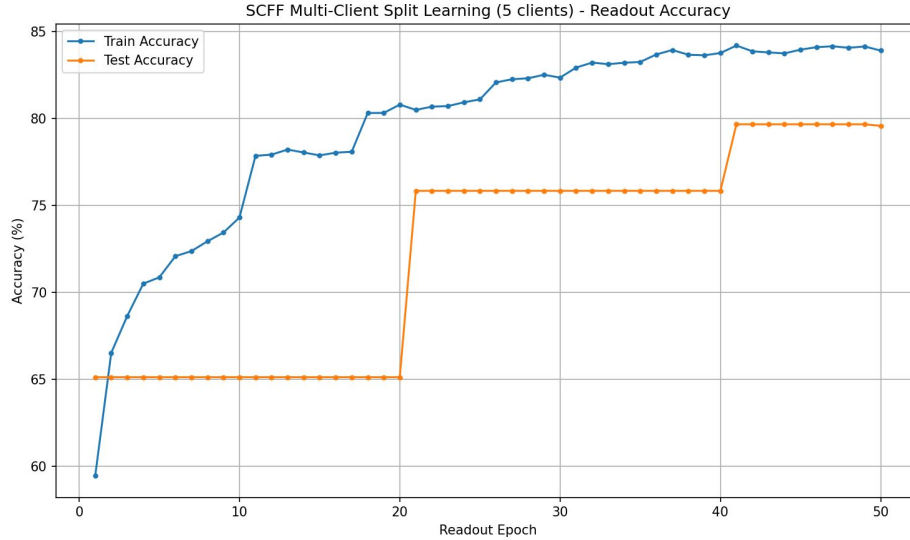


Figure 4: Linear readout accuracy for the 10-client sequential relay. Each client holds 5,000 training samples. Test accuracy remains close to the single-client result.

5.3 Fixed Cut Point

The current implementation always cuts after Layer 0. In practice it might be useful to tune the cut point as a hyperparameter, or to support multi-hop splits where more than two devices share layers. SCFF’s per-layer loss makes this a natural extension: any `.detach()` call between consecutive layers is a valid split boundary.

5.4 Non-IID Correction Is Missing

The relay does not include any mechanism to correct for distribution shift across clients. Proximal terms [Li et al., 2020] or control variates [Karimireddy et al., 2020] could reduce the per-client drift and likely close some of the non-IID accuracy gap. Applying these ideas to the SCFF relay, where the “global model” is the set of Layer 0 weights being passed between clients, seems straightforward in principle.

5.5 Linear Probe May Understate Representation Quality

The evaluation protocol trains a linear classifier on frozen features. This is standard in self-supervised learning but it only measures linear separability. Fine-tuning the whole network or using a nonlinear probe might reveal richer structure in the representations, and could produce higher accuracy numbers. The 50-epoch readout schedule with a fixed Adam optimizer is also not optimized; a longer or differently scheduled readout might close the gap between the linear probe results and what the features actually encode.

5.6 Scale Beyond CIFAR-10

The architecture is tuned for 32×32 images. Scaling to STL-10 (96×96) or ImageNet (224×224) would need larger kernels, adjusted pooling strides, and more layers. The `config.json` already includes STL-10 and ImageNet configurations, which suggests these were at least considered. Whether the SCFF loss hyperparameters transfer cleanly to those scales without significant retuning is unclear.

5.7 Theoretical Guarantees

FF-based methods, including SCFF, do not have the convergence guarantees that exist for standard backpropagation under the neural tangent kernel framework. The conditions under which greedy layer-local contrastive training converges to useful representations, and how the split topology affects convergence rate, are open questions. Connecting SCFF’s empirical performance to a formal analysis would strengthen the case for using it in safety-critical applications.

Conclusion

This project shows that the SCFF objective and split learning are a natural match. Because SCFF already trains each layer independently with a local loss, the physical split boundary introduces no new constraints and no accuracy penalty. The sequential multi-client relay extends this to distributed data while staying competitive in accuracy.

The key practical takeaway is that gradient-free split learning is achievable without sacrificing the learning signal: raw data stays on the client, no gradients cross any boundary, and the network still learns features that support 79–80% test accuracy on CIFAR-10 with a linear probe.

The repository is at <https://github.com/MShayan-Ejaz/SCFF-Split-Learning>.

References

- Martin Abadi, Andy Chu, Ian Goodfellow, et al. Deep learning with differential privacy. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 308–318, 2016.
- Yoshua Bengio, Pascal Lamblin, Dan Popovici, and Hugo Larochelle. Greedy layer-wise training of deep networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 19, 2007.
- Mathilde Caron, Hugo Touvron, Ishan Misra, et al. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9650–9660, 2021.

- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020.
- Jean-Bastien Grill, Florian Strub, Florent Alché, et al. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 21271–21284, 2020.
- Zecheng He, Tianwei Zhang, and Ruby B. Lee. Model inversion attacks against collaborative inference. In *Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC)*, pages 148–162, 2019.
- Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.
- Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 5132–5143, 2020.
- Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. In *Technical Report, University of Toronto*, 2009.
- Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Alexander Smola, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 2, pages 429–450, 2020.
- Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.
- Arild Nøkland and Lars Haakon Eidnes. Training neural networks with local error signals. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 4839–4850, 2019.
- Dario Pasquini, Giuseppe Ateniese, and Massimo Bernaschi. Unleashing the tiger: Inference attacks on split learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2113–2129, 2021.
- Chandra Thapa, Seyit Ragib Chhetri, Antonios Ignatiadis, et al. SplitFed: When federated learning meets split learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8485–8493, 2022.
- Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564*, 2018.

A Source Code

This appendix contains the complete source code for all three scripts and the configuration file. The code is also available at <https://github.com/MShayan-Ejaz/SCFF-Split-Learning>.

A.1 config.json — Architecture and Optimizer Configuration

```
1 {
2   "CIFAR":{
3     "layer_configs": [
4       {
5         "num": 1, "padding_mode": "reflect", "pad": 2,
6         "ch_in": 3, "channels": 96, "kernel_size": 5,
7         "padding": 1, "pooltype": "Max",
8         "pool_size": 4, "stride_size": 2,
9         "extra_pool_size": 2, "concat": [1], "act": "triangle"
10      },
11      {
12        "num": 2, "padding_mode": "reflect", "pad": 1,
13        "ch_in": 96, "channels": 384, "kernel_size": 3,
14        "padding": 1, "pooltype": "Max",
15        "pool_size": 4, "stride_size": 2,
16        "extra_pool_size": 2, "concat": [1,0], "act": "triangle"
17      },
18      {
19        "num": 3, "padding_mode": "reflect", "pad": 1,
20        "ch_in": 384, "channels": 1536, "kernel_size": 3,
21        "padding": 0, "pooltype": "Avg",
22        "pool_size": 2, "stride_size": 2,
23        "extra_pool_size": 2, "concat": [1,1,1], "act": "relu"
24      }
25    ],
26    "opt_configs": [
27      {"lr": 0.01, "weight_decay": 0.0001, "gamma": 0.7,
28       "th1": 0, "th2": 1, "lamda": 0.0012, "epochs": 6},
29      {"lr": 0.002, "weight_decay": 0.0001, "gamma": 0.8,
30       "th1": 5, "th2": 9, "lamda": 0.0007, "epochs": 4},
31      {"lr": 0.0002, "weight_decay": 0.0003, "gamma": 1.0,
32       "th1": 6, "th2": 10, "lamda": 0.0005, "epochs": 21}
33    ]
34  }
35 }
```

Listing 1: config.json — layer and optimizer hyperparameters for CIFAR-10 (and other datasets).

A.2 SCFF_CIFAR_Split.py — Single-Client Split Learning

```
1 """
2 SCFF CIFAR-10 Split Learning Implementation
3 =====
4 Split point: After Layer 0 (client). Server runs Layers 1-2.
5 Mathematically identical to baseline due to greedy layer-wise training (.detach()).
6 """
7
8 import os
9 os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE" # Fix OpenMP duplicate runtime on Windows
10
11 import torch
12 import torch.nn as nn
13 import torch.optim as optim
14 from torch.optim import Adam, AdamW
```

```

15 from torch.optim.lr_scheduler import ExponentialLR, StepLR, LinearLR
16
17 import torch.nn.functional as F
18 import torch.nn.init as init
19 import matplotlib
20 matplotlib.use('Agg')
21 import matplotlib.pyplot as plt
22
23 import torchvision
24 from torchvision.transforms import transforms, ToPILImage
25 from torch.utils.data import TensorDataset, DataLoader, Dataset, random_split, Subset
26 import argparse
27 import time
28
29 import numpy as np
30 from numpy import fft
31 import math
32 import json
33
34 # =====
35 # Reproducibility
36 # =====
37 torch.manual_seed(1234)
38
39 # =====
40 # Transforms (unchanged from baseline)
41 # =====
42 s = 0.5
43 transform1 = transforms.Compose([
44     transforms.RandomResizedCrop(size=(32, 32), scale=(0.8, 1.0), ratio=(0.75, 1.33)),
45     transforms.RandomHorizontalFlip(),
46     transforms.RandomApply([transforms.ColorJitter(brightness=0.8*s, contrast=0.8*s,
47         saturation=0.8*s, hue=0.2*s)], p=0.8),
48     transforms.ToTensor(),
49     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
50 ])
51 transform2 = transforms.Compose([
52     transforms.RandomResizedCrop(size=(32, 32), scale=(0.8, 1.0), ratio=(0.75, 1.33)),
53     transforms.RandomHorizontalFlip(),
54     transforms.RandomApply([transforms.ColorJitter(brightness=0.8*s, contrast=0.8*s,
55         saturation=0.8*s, hue=0.2*s)], p=0.8),
56     transforms.ToTensor(),
57     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
58 ])
59 transform_train = transforms.Compose([
60     transforms.RandomHorizontalFlip(),
61     transforms.ToTensor(),
62     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
63 ])
64
65 transform_test = transforms.Compose([
66     transforms.ToTensor(),
67     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
68 ])
69
70 # =====
71 # Dataset classes (unchanged from baseline)
72 # =====
73 class DualAugmentCIFAR10(torchvision.datasets.CIFAR10):
74     def __init__(self, root, augment="No", *args, **kwargs):
75         super(DualAugmentCIFAR10, self).__init__(root, *args, **kwargs)
76         self.augment = augment
77
78     def __getitem__(self, index):
79         img, target = self.data[index], self.targets[index]
80         img_pil = ToPILImage()(img)

```

```

81         img_original = transform_train(img_pil)
82
83         if self.augment == "single":
84             img1 = transform1(img_pil)
85             return img_original, img1, img_original, target
86         elif self.augment == "dual":
87             img1 = transform1(img_pil)
88             img2 = transform2(img_pil)
89             return img_original, img1, img2, target
90         else:
91             return img_original, target
92
93
94 class DualAugmentCIFAR10_test(torchvision.datasets.CIFAR10):
95     def __init__(self, aug=False, *args, **kwargs):
96         super(DualAugmentCIFAR10_test, self).__init__(*args, **kwargs)
97         self.aug = aug
98
99     def __getitem__(self, index):
100         img, target = self.data[index], self.targets[index]
101         img = ToPILImage()(img)
102         if self.aug:
103             img = transform_train(img)
104         else:
105             img = transform_test(img)
106         return img, target
107
108
109 def get_train(batchsize, augment, Factor):
110     torch.manual_seed(1234)
111     trainset = DualAugmentCIFAR10(root='./data', train=True, download=True, augment=
        augment)
112     sup_trainset = DualAugmentCIFAR10_test(root='./data', aug=True, train=True, download
        =True)
113     factor = Factor
114     train_len = int(len(trainset) * factor)
115     indices = torch.randperm(len(trainset)).tolist()
116     train_indices = indices[:train_len]
117     val_indices = indices[train_len:]
118     train_data = Subset(trainset, train_indices)
119     sup_train_data = Subset(sup_trainset, train_indices)
120     val_data = Subset(sup_trainset, val_indices)
121     testset = DualAugmentCIFAR10_test(root='./data', aug=False, train=False, download=
        True)
122     testloader = DataLoader(testset, batch_size=1000, shuffle=False, num_workers=2)
123     trainloader = DataLoader(train_data, batch_size=batchsize, shuffle=True, num_workers
        =2)
124     if factor == 1:
125         valloader = testloader
126     else:
127         valloader = DataLoader(val_data, batch_size=1000, shuffle=True, num_workers=2)
128     sup_trainloader = DataLoader(sup_train_data, batch_size=64, shuffle=True)
129     return trainloader, valloader, testloader, sup_trainloader
130
131 # =====
132 # Utility functions (unchanged from baseline)
133 # =====
134 def get_pos_neg_batch_imgcats(batch_pos1, batch_pos2, p=1):
135     batch_size = len(batch_pos1)
136     batch_pos = torch.cat((batch_pos1, batch_pos2), dim=1)
137     random_indices = (torch.randperm(batch_size - 1) + 1)[:min(p, batch_size - 1)]
138     labels = torch.arange(batch_size)
139     batch_negs = []
140     for i in random_indices:
141         batch_neg = batch_pos2[(labels + i) % batch_size]
142         batch_neg = torch.cat((batch_pos1, batch_neg), dim=1)
143         batch_negs.append(batch_neg)
144     return batch_pos, torch.cat(batch_negs)

```

```

145
146
147 def stdnorm(x, dims=[1, 2, 3]):
148     x = x - torch.mean(x, dim=(dims), keepdim=True)
149     x = x / (1e-10 + torch.std(x, dim=(dims), keepdim=True))
150     return x
151
152
153 class standardnorm(nn.Module):
154     def __init__(self, dims=[1, 2, 3]):
155         super(standardnorm, self).__init__()
156         self.dims = dims
157
158     def forward(self, x):
159         x = x - torch.mean(x, dim=(self.dims), keepdim=True)
160         x = x / (1e-10 + torch.std(x, dim=(self.dims), keepdim=True))
161         return x
162
163
164 class L2norm(nn.Module):
165     def __init__(self, dims=[1, 2, 3]):
166         super(L2norm, self).__init__()
167         self.dims = dims
168
169     def forward(self, x):
170         return x / (x.norm(p=2, dim=(self.dims), keepdim=True) + 1e-10)
171
172
173 class triangle(nn.Module):
174     def __init__(self):
175         super(triangle, self).__init__()
176
177     def forward(self, x):
178         x = x - torch.mean(x, axis=1, keepdims=True)
179         return F.relu(x)
180
181
182 # =====
183 # Conv2d layer (unchanged from baseline)
184 # =====
185 class Conv2d(nn.Module):
186     def __init__(self, input_channels, output_channels, kernel_size, pad=0,
187                 batchnorm=False, normdims=[1, 2, 3], norm="stdnorm", bias=True,
188                 dropout=0.0, padding_mode="reflect", concat=True, act="relu"):
189         super(Conv2d, self).__init__()
190         self.input_channels = input_channels
191         self.output_channels = output_channels
192         self.kernel_size = kernel_size
193         self.normdims = normdims
194         self.concat = concat
195         self.relu = torch.nn.ReLU()
196         self.conv_layer = nn.Conv2d(in_channels=input_channels, out_channels=
197                                     output_channels,
198                                     kernel_size=kernel_size, bias=bias)
199         init.xavier_uniform_(self.conv_layer.weight)
200         self.padding_mode = padding_mode
201         self.F_padding = (pad, pad, pad, pad)
202         if act == 'relu':
203             self.act = torch.nn.ReLU()
204         else:
205             self.act = triangle()
206         if batchnorm:
207             self.bn1 = nn.BatchNorm2d(self.input_channels, affine=False)
208         else:
209             self.bn1 = nn.Identity()
210         if norm == "L2norm":
211             self.norm = L2norm(dims=normdims)
212         elif norm == "stdnorm":

```

```

212         self.norm = standardnorm(dims=normdims)
213     else:
214         self.norm = nn.Identity()
215
216     def forward(self, x):
217         x = self.bn1(x)
218         x = F.pad(x, self.F_padding, self.padding_mode)
219         x = self.norm(x)
220         if self.concat:
221             lenchannel = x.size(1) // 2
222             out = self.conv_layer(x[:, :lenchannel]) + self.conv_layer(x[:, lenchannel
                :])
223         else:
224             out = self.conv_layer(x)
225         return out
226
227
228 # =====
229 # Scheduler and evaluation helpers (unchanged from baseline)
230 # =====
231 class CustomStepLR(StepLR):
232     def __init__(self, optimizer, nb_epochs):
233         threshold_ratios = [0.2, 0.35, 0.5, 0.6, 0.7, 0.8, 0.9]
234         self.step_threshold = [int(nb_epochs * r) for r in threshold_ratios]
235         super().__init__(optimizer, -1, False)
236
237     def get_lr(self):
238         if self.last_epoch in self.step_threshold:
239             return [group['lr'] * 0.5 for group in self.optimizer.param_groups]
240         return [group['lr'] for group in self.optimizer.param_groups]
241
242
243 class EvaluationConfig:
244     def __init__(self, device, dims, dims_in, dims_out, stdnorm_out, out_dropout,
245                 Layer_out, pre_std, all_neurons):
246         self.device = device
247         self.dims = dims
248         self.dims_in = dims_in
249         self.dims_out = dims_out
250         self.stdnorm_out = stdnorm_out
251         self.out_dropout = out_dropout
252         self.Layer_out = Layer_out
253         self.all_neurons = all_neurons
254         self.pre_std = pre_std
255
256
257 def calculate_output_length(dims, nets, extra_pool, Layer, all_neurons):
258     lengths = 0
259     if all_neurons:
260         for i, length in enumerate(dims):
261             if i in Layer:
262                 lengths += length
263     else:
264         for i, length in enumerate(dims):
265             if i in Layer:
266                 len_after_pool = math.ceil(
267                     (math.sqrt(length / nets[i].output_channels) - extra_pool[i].
268                      kernel_size)
269                     / extra_pool[i].stride + 1)
270                 lengths += len_after_pool * len_after_pool * nets[i].output_channels
271     return lengths
272
273 def build_classifier(lengths, config):
274     classifier = nn.Sequential(
275         nn.Dropout(config.out_dropout),
276         nn.Linear(lengths, 10)
277     ).to(config.device)

```

```

278     if torch.cuda.device_count() > 2:
279         classifier = nn.DataParallel(classifier)
280     return classifier
281
282
283 def create_layer(layer_config, opt_config, load_params, device, act):
284     layer_num = layer_config['num'] - 1
285     net = Conv2d(layer_config["ch_in"], layer_config["channels"],
286                 (layer_config["kernel_size"], layer_config["kernel_size"]),
287                 pad=layer_config["pad"], norm="stdnorm",
288                 padding_mode=layer_config["padding_mode"], act=act)
289     if load_params:
290         net.load_state_dict(torch.load('./results/params_CIFAR_1' + str(layer_num) + '.
291                                    .pth', map_location='cpu'))
292         for param in net.parameters():
293             param.requires_grad = False
294     if layer_config["pooltype"] == 'Avg':
295         pool = nn.AvgPool2d(kernel_size=layer_config["pool_size"],
296                             stride=layer_config["stride_size"],
297                             padding=layer_config["padding"], ceil_mode=True)
298     else:
299         pool = nn.MaxPool2d(kernel_size=layer_config["pool_size"],
300                             stride=layer_config["stride_size"],
301                             padding=layer_config["padding"], ceil_mode=True)
302     extra_pool = nn.AvgPool2d(kernel_size=layer_config["extra_pool_size"],
303                               stride=layer_config["extra_pool_size"],
304                               padding=0, ceil_mode=True)
305     net.to(device)
306     optimizer = AdamW(net.parameters(), lr=opt_config["lr"], weight_decay=opt_config["
307                       weight_decay"])
308     scheduler = ExponentialLR(optimizer, opt_config["gamma"])
309     return net, pool, extra_pool, optimizer, scheduler
310
311 # =====
312 # SplitClient: Holds Layer 0
313 # =====
314 class SplitClient:
315     """Client holds Layer 0 and trains it on raw images."""
316
317     def __init__(self, net, pool, extra_pool, optimizer, scheduler, device):
318         self.net = net
319         self.pool = pool
320         self.extra_pool = extra_pool
321         self.optimizer = optimizer
322         self.scheduler = scheduler
323         self.device = device
324
325     def train_step(self, x, threshold1, threshold2, a, b, lamda, p=1):
326         """
327         Train Layer 0 on a batch.
328
329         Args:
330             x: raw images [B, C, H, W]
331             threshold1: positive threshold for layer 0
332             threshold2: negative threshold for layer 0
333             a, b: loss scaling
334             lamda: regularization coefficient for layer 0
335             p: number of negative samples
336
337         Returns:
338             x_pos_pooled: detached pooled activated pos output [B, 96, H', W']
339             x_neg_pooled: detached pooled activated neg output [B*p, 96, H', W']
340             goodness_pos_mean: scalar for logging
341             goodness_neg_mean: scalar for logging
342         """
343         self.net.train()

```

```

344     # Layer 0 has concat=True, so stdnorm + pos/neg creation
345     x = stdnorm(x, dims=[1, 2, 3])
346     x_pos, x_neg = get_pos_neg_batch_imgcats(x, x, p=p)
347
348     out_pos = self.net(x_pos)
349     out_neg = self.net(x_neg)
350
351     goodness_pos = self.net.relu(out_pos).pow(2).mean([1])
352     goodness_neg = self.net.relu(out_neg).pow(2).mean([1])
353
354     # SCFF softplus loss (identical to baseline)
355     self.optimizer.zero_grad()
356     loss = (torch.log(1 + torch.exp(a * (-goodness_pos + threshold1))).mean([1, 2]).
357             + torch.log(1 + torch.exp(b * (goodness_neg - threshold2))).mean([1, 2])
358             .mean()
359             + lamda * torch.norm(goodness_pos, p=2, dim=(1, 2)).mean())
360     loss.backward()
361     self.optimizer.step()
362
363     # Detach and pool for sending to server
364     x_pos_pooled = self.pool(self.net.act(out_pos)).detach()
365     x_neg_pooled = self.pool(self.net.act(out_neg)).detach()
366
367     gp = torch.mean(goodness_pos.mean([1, 2])).item()
368     gn = torch.mean(goodness_neg.mean([1, 2])).item()
369
370     return x_pos_pooled, x_neg_pooled, gp, gn
371
372 def extract_features(self, x, dims_in, dims_out, stdnorm_out, Layer_out):
373     """
374     Inference: extract Layer 0 features for evaluation.
375
376     Returns:
377         x_after_pool: activation after pool (to send to server)
378         outputs: list of flattened features if layer 0 is in Layer_out
379     """
380     self.net.eval()
381     outputs = []
382
383     with torch.no_grad():
384         # Layer 0 has concat=True
385         x = stdnorm(x, dims=dims_in)
386         x = torch.cat((x, x), dim=1)
387         x = self.pool(self.net.act(self.net(x)))
388
389         out = self.extra_pool(x)
390         if stdnorm_out:
391             out = stdnorm(out, dims=dims_out)
392             out = out.flatten(start_dim=1)
393         if 0 in Layer_out:
394             outputs.append(out)
395
396     return x, outputs
397
398 # =====
399 # SplitServer: Holds Layers 1-2
400 # =====
401 class SplitServer:
402     """Server holds Layers 1..NL-1 and trains them on activations from client."""
403
404     def __init__(self, nets, pools, extra_pools, optimizers, schedulers, device):
405         """
406         Args:
407             nets: list of Conv2d for layers 1..NL-1
408             pools, extra_pools, optimizers, schedulers: corresponding lists
409         """

```

```

410     self.nets = nets
411     self.pools = pools
412     self.extra_pools = extra_pools
413     self.optimizers = optimizers
414     self.schedulers = schedulers
415     self.device = device
416
417     def train_step(self, x, x_neg, threshold1_list, threshold2_list, a, b, lamda_list,
418                   alleps, epoch, p=1):
419         """
420         Train server layers on received activations.
421
422         Args:
423             x: pos activations from client [B, C, H, W]
424             x_neg: neg activations from client [B*p, C, H, W]
425             threshold1_list: thresholds for server layers (indexed 0=layer1, 1=layer2,
426                             ...)
427             threshold2_list: same for neg
428             alleps: list of max epochs per server layer
429             epoch: current epoch
430             p: number of negative samples
431
432         Returns:
433             goodness_pos_mean, goodness_neg_mean: from last layer, for logging
434         """
435         gp, gn = 0, 0
436         for i, net in enumerate(self.nets):
437             net.train()
438
439             if net.concat:
440                 # concat=True layers: stdnorm then create new pos/neg pairs
441                 x = stdnorm(x, dims=[1, 2, 3])
442                 x, x_neg = get_pos_neg_batch_imgcats(x, x, p=p)
443
444             out_pos = net(x)
445             out_neg = net(x_neg)
446
447             goodness_pos = net.relu(out_pos).pow(2).mean([1])
448             goodness_neg = net.relu(out_neg).pow(2).mean([1])
449
450             if epoch < alleps[i]:
451                 self.optimizers[i].zero_grad()
452                 loss = (torch.log(1 + torch.exp(a * (-goodness_pos + threshold1_list[i]
453                 )))
454                        + torch.log(1 + torch.exp(b * (goodness_neg - threshold2_list[i]
455                        )))
456                        + lamda_list[i] * torch.norm(goodness_pos, p=2, dim=(1, 2)).mean(
457                        ))
458                 loss.backward()
459                 self.optimizers[i].step()
460
461             # Detach and pool before passing to next layer
462             x = self.pools[i](net.act(out_pos)).detach()
463             x_neg = self.pools[i](net.act(out_neg)).detach()
464
465             gp = torch.mean(goodness_pos.mean([1, 2])).item()
466             gn = torch.mean(goodness_neg.mean([1, 2])).item()
467
468         return gp, gn
469
470     def extract_features(self, x, dims_in, dims_out, stdnorm_out, Layer_out):
471         """
472         Inference: extract features from server layers for evaluation.
473
474         Args:
475             x: activation from client's Layer 0 output

```

```

473         Layer_out: which global layer indices to include (server layers are indices
474             1, 2, ...)
475
476     Returns:
477         outputs: list of flattened features for layers in Layer_out
478     """
479     outputs = []
480     for i, net in enumerate(self.nets):
481         net.eval()
482         with torch.no_grad():
483             global_layer_idx = i + 1 # server layer 0 = global layer 1
484
485             if net.concat:
486                 x = stdnorm(x, dims=dims_in)
487                 x = torch.cat((x, x), dim=1)
488
489             x = self.pools[i](net.act(net(x)))
490
491             out = self.extra_pools[i](x)
492             if stdnorm_out:
493                 out = stdnorm(out, dims=dims_out)
494             out = out.flatten(start_dim=1)
495             if global_layer_idx in Layer_out:
496                 outputs.append(out)
497
498     return outputs
499
500 # =====
501 # SplitTrainer: Orchestrates training and evaluation
502 # =====
503 class SplitTrainer:
504     """Orchestrates split learning between client and server."""
505
506     def __init__(self, client, server, loaders, eval_config, threshold1, threshold2,
507         lamda, period, alleps, p=1, a=1, b=1):
508         self.client = client
509         self.server = server
510         self.loaders = loaders
511         self.eval_config = eval_config
512
513         # threshold1/2 and lamda: index 0 = client (layer 0), indices 1+ = server layers
514         self.threshold1 = threshold1
515         self.threshold2 = threshold2
516         self.lamda = lamda
517         self.period = period
518         self.alleps = alleps
519         self.p = p
520         self.a = a
521         self.b = b
522
523         self.all_pos = [[] for _ in range(len(threshold1))]
524         self.all_neg = [[] for _ in range(len(threshold1))]
525         self.Dims = []
526
527     def train(self, epochs):
528         """Run the full SCFF training loop."""
529         trainloader = self.loaders[0]
530         device = self.eval_config.device
531         nbbatches = 0
532         firstpass = True
533
534         for epoch in range(epochs):
535             print(f"Epoch {epoch}")
536             self.client.net.train()
537             for net in self.server.nets:
538                 net.train()
539

```

```

540     goodness_pos_accum = 0
541     goodness_neg_accum = 0
542
543     for numbatches, (x, _) in enumerate(trainloader):
544         nbatches += 1
545         x = x.to(device)
546
547         # --- Client: Layer 0 ---
548         x_pos_pooled, x_neg_pooled, gp_client, gn_client = self.client.
            train_step(
549             x, self.threshold1[0], self.threshold2[0],
550             self.a, self.b, self.lamda[0], p=self.p
551         )
552
553         if firstpass:
554             _, ch, h, w = x_pos_pooled.shape
555             self.Dims.append(ch * h * w)
556             print(f"Layer 0 : x.shape: {x_pos_pooled.shape} y.shape (after MaxP)
                : {x_pos_pooled.shape}", end=" ")
557
558         # Client scheduler
559         if epoch < self.alleps[0] and (nbatches + 1) % self.period[0] == 0:
560             self.client.scheduler.step()
561             print(f'nbatches {nbatches + 1} learning rate: {self.client.
                scheduler.get_last_lr()[0]}')
562
563         # --- Server: Layers 1+ ---
564         # Server thresholds/lamda are indices 1+ in the global lists
565         server_th1 = self.threshold1[1:]
566         server_th2 = self.threshold2[1:]
567         server_lamda = self.lamda[1:]
568         server_alleps = self.alleps[1:]
569
570         gp_server, gn_server = self.server.train_step(
571             x_pos_pooled, x_neg_pooled,
572             server_th1, server_th2,
573             self.a, self.b, server_lamda,
574             server_alleps, epoch, p=self.p
575         )
576
577         if firstpass:
578             # Compute dims for server layers by running a dummy forward
579             dummy_x = x_pos_pooled
580             for si, snet in enumerate(self.server.nets):
581                 with torch.no_grad():
582                     if snet.concat:
583                         dummy_x = stdnorm(dummy_x, dims=[1, 2, 3])
584                         dummy_x = torch.cat((dummy_x, dummy_x), dim=1)
585                     dummy_x = self.server.pools[si](snet.act(snet(dummy_x)))
586                     _, ch, h, w = dummy_x.shape
587                     self.Dims.append(ch * h * w)
588                     print(f"Layer {si + 1} : x.shape: {dummy_x.shape} y.shape (
                        after MaxP): {dummy_x.shape}", end=" ")
589             print()
590             firstpass = False
591
592         # Server schedulers
593         for si in range(len(self.server.nets)):
594             if epoch < self.alleps[si + 1] and (nbatches + 1) % self.period[si
                + 1] == 0:
595                 self.server.schedulers[si].step()
596                 print(f'nbatches {nbatches + 1} learning rate: {self.server.
                    schedulers[si].get_last_lr()[0]}')
597
598         goodness_pos_accum += gp_server
599         goodness_neg_accum += gn_server
600
601         if numbatches == len(trainloader) - 1:

```

```

602         print(goodness_pos_accum / len(trainloader), goodness_neg_accum /
603               len(trainloader))
604         # Log for last layer (global index NL-1)
605         nl = len(self.threshold1)
606         self.all_pos[nl - 1].append(goodness_pos_accum)
607         self.all_neg[nl - 1].append(goodness_neg_accum)
608         goodness_pos_accum, goodness_neg_accum = 0, 0
609
610     print("Training done..")
611     return self.Dims
612
613 def evaluate(self):
614     """
615     Evaluate by training a linear readout on features from all layers.
616
617     Returns:
618         acc_train, acc_val: final training and validation accuracy
619     """
620     config = self.eval_config
621     _, valloader, testloader, suptrloader = self.loaders
622
623     current_rng_state = torch.get_rng_state()
624     torch.manual_seed(42)
625
626     # Build full nets/pools/extra_pools lists for calculate_output_length
627     all_nets = [self.client.net] + self.server.nets
628     all_extra_pools = [self.client.extra_pool] + self.server.extra_pools
629
630     lengths = calculate_output_length(self.Dims, all_nets, all_extra_pools,
631                                     config.Layer_out, config.all_neurons)
632     print(f"Classifier input length: {lengths}")
633     classifier = build_classifier(lengths, config)
634
635     optimizer = optim.Adam(classifier.parameters(), lr=0.001)
636     lr_scheduler = CustomStepLR(optimizer, nb_epochs=50)
637     criterion = nn.CrossEntropyLoss()
638
639     # Use test set for validation (search=False in baseline default)
640     eval_valloader = testloader
641
642     # Set all nets to eval
643     self.client.net.eval()
644     for net in self.server.nets:
645         net.eval()
646
647     train_accs = []
648     test_accs = []
649
650     for epoch_eval in range(50):
651         # --- Train readout ---
652         classifier.train()
653         correct = 0
654         total = 0
655         for x, labels in suptrloader:
656             x = x.to(config.device)
657             labels = labels.to(config.device)
658
659             with torch.no_grad():
660                 x_after_client, client_outputs = self.client.extract_features(
661                     x, config.dims_in, config.dims_out, config.stdnorm_out, config.
662                     Layer_out)
663                 server_outputs = self.server.extract_features(
664                     x_after_client, config.dims_in, config.dims_out, config.
665                     stdnorm_out, config.Layer_out)
666
667             outputs = client_outputs + server_outputs
668             outputs = torch.cat(outputs, dim=1)

```

```

667         optimizer.zero_grad()
668         preds = classifier(outputs)
669         loss = criterion(preds, labels)
670         loss.backward()
671         optimizer.step()
672
673         _, predicted = torch.max(preds.data, 1)
674         total += labels.size(0)
675         correct += (predicted == labels).sum().item()
676
677     train_acc = correct / total
678     lr_scheduler.step()
679
680     if epoch_eval % 20 == 0 or epoch_eval == 49:
681         print(f'Accuracy of the network on the 50000 train images: {100 *
682               train_acc} %')
683
684         # Compute clean train accuracy
685         train_acc_clean = self._test_readout(classifier, suptrloader, criterion,
686                                              config, epoch_eval, 'Train')
687         test_acc = self._test_readout(classifier, eval_valloader, criterion,
688                                      config, epoch_eval, 'Val')
689
690         # Record every epoch for plotting
691         train_accs.append(train_acc)
692         # For test, only update on eval epochs, else repeat last
693         if epoch_eval % 20 == 0 or epoch_eval == 49:
694             last_test_acc = test_acc
695         test_accs.append(last_test_acc if 'last_test_acc' in dir() else 0)
696
697     # Final evaluation
698     final_train = self._test_readout(classifier, suptrloader, criterion, config, 49,
699                                     'Train')
700     final_test = self._test_readout(classifier, eval_valloader, criterion, config,
701                                    49, 'Val')
702
703     torch.set_rng_state(current_rng_state)
704
705     # Plot
706     self._plot_accuracy(train_accs, test_accs)
707
708     return final_train, final_test
709
710 def _test_readout(self, classifier, loader, criterion, config, epoch, mode):
711     """Test readout classifier on a loader."""
712     classifier.eval()
713     correct = 0
714     total = 0
715     running_loss = 0.0
716
717     with torch.no_grad():
718         for x, labels in loader:
719             x = x.to(config.device)
720             labels = labels.to(config.device)
721
722             x_after_client, client_outputs = self.client.extract_features(
723                 x, config.dims_in, config.dims_out, config.stdnorm_out, config.
724                 Layer_out)
725             server_outputs = self.server.extract_features(
726                 x_after_client, config.dims_in, config.dims_out, config.stdnorm_out,
727                 config.Layer_out)
728
729             outputs = client_outputs + server_outputs
730             outputs = torch.cat(outputs, dim=1)
731             preds = classifier(outputs)
732
733             _, predicted = torch.max(preds.data, 1)
734             total += labels.size(0)

```

```

728         correct += (predicted == labels).sum().item()
729         loss = criterion(preds, labels)
730         running_loss += loss.item()
731
732     acc = correct / total
733     if mode == 'Val':
734         print(f'Accuracy of the network on the 10000 {mode} images: {100 * acc} %')
735         print(f'[{epoch + 1}] loss: {running_loss / total:.3f}')
736
737     classifier.train()
738     return acc
739
740 def _plot_accuracy(self, train_accs, test_accs):
741     """Save epoch-vs-accuracy plot."""
742     plt.figure(figsize=(10, 6))
743     epochs = list(range(1, len(train_accs) + 1))
744     plt.plot(epochs, [a * 100 for a in train_accs], label='Train Accuracy', marker='
745     plt.plot(epochs, [a * 100 for a in test_accs], label='Test Accuracy', marker='
746     plt.xlabel('Readout Epoch')
747     plt.ylabel('Accuracy (%)')
748     plt.title('SCFF Split Learning - Linear Readout Accuracy')
749     plt.legend()
750     plt.grid(True)
751     plt.tight_layout()
752     plt.savefig('/home/claw/.openclaw/workspace/scff_split_accuracy.png', dpi=150)
753     plt.close()
754     print("Plot saved to scff_split_accuracy.png")
755
756 # =====
757 # Pre-training validation
758 # =====
759
760 def run_validation_checks(client, server, device, trainloader, config_json):
761     """Run pre-training validation checks."""
762     print("=" * 60)
763     print("Running pre-training validation checks...")
764     print("=" * 60)
765
766     # 1. Config integrity
767     required_keys = ['CIFAR']
768     for key in required_keys:
769         assert key in config_json, f"Missing config key: {key}"
770     assert 'layer_configs' in config_json['CIFAR'], "Missing 'layer_configs' in config"
771     assert 'opt_configs' in config_json['CIFAR'], "Missing 'opt_configs' in config"
772     print("[OK] Config integrity check passed")
773
774     # 2. Input tensor dimensions
775     x_sample, _ = next(iter(trainloader))
776     x_sample = x_sample.to(device)
777     assert x_sample.shape[1:] == (3, 32, 32), \
778         f"Expected input shape (B, 3, 32, 32), got {x_sample.shape}"
779     print(f"[OK] Input tensor shape: {x_sample.shape}")
780
781     # 3. Channel consistency at split boundary
782     client_out_channels = client.net.output_channels
783     server_in_channels = server.nets[0].input_channels
784     assert client_out_channels == server_in_channels, \
785         f"Channel mismatch at split: client outputs {client_out_channels}, server
786     print(f"[OK] Channel consistency: client out={client_out_channels}, server in={
787         server_in_channels}")
788
789     # 4. Split activation shapes
790     with torch.no_grad():
791         client.net.eval()
792         x_test = stdnorm(x_sample[:4], dims=[1, 2, 3])

```

```

792     x_pos_test, x_neg_test = get_pos_neg_batch_imgcats(x_test, x_test, p=1)
793     out_test = client.net(x_pos_test)
794     x_pooled_test = client.pool(client.net.act(out_test))
795     print(f"[OK] Client output shape after pool: {x_pooled_test.shape}")
796     assert x_pooled_test.shape[1] == client_out_channels, \
797         f"Pooled output channels {x_pooled_test.shape[1]} != expected {
            client_out_channels}"
798
799 # 5. Client/server forward compatibility (dummy batch)
800 with torch.no_grad():
801     # Run through server layers
802     dummy = x_pooled_test
803     for i, net in enumerate(server.nets):
804         net.eval()
805         if net.concat:
806             dummy = stdnorm(dummy, dims=[1, 2, 3])
807             dummy = torch.cat((dummy, dummy), dim=1)
808             dummy = server.pools[i](net.act(net(dummy)))
809     print(f"[OK] Server final output shape: {dummy.shape}")
810
811 # 6. Label alignment
812 for x_batch, labels_batch in trainloader:
813     assert x_batch.shape[0] == labels_batch.shape[0], \
814         f"Batch size mismatch: images {x_batch.shape[0]}, labels {labels_batch.shape
            [0]}"
815     break
816 print("[OK] Label alignment check passed")
817
818 # 7. NL check for split
819 NL = 1 + len(server.nets)
820 if NL < 2:
821     raise ValueError(f"Need at least 2 layers for split learning, got NL={NL}")
822 print(f"[OK] NL={NL} is valid for split learning")
823
824 print("=" * 60)
825 print("All validation checks passed!")
826 print("=" * 60)
827
828
829 # =====
830 # Main entry point
831 # =====
832 def get_arguments():
833     parser = argparse.ArgumentParser(description="SCFF Split Learning Training Script",
834                                     add_help=False)
835     parser.add_argument("--lr", nargs='+', type=float, default=[0.02, 0.001, 0.0004])
836     parser.add_argument("--gamma", nargs='+', type=float, default=[0.99, 0.9, 0.99])
837     parser.add_argument("--period", nargs='+', type=int, default=[500, 500, 500])
838     parser.add_argument("--weight_decay", nargs='+', type=float, default=[0.0001,
839                                     0.0003, 0.0001])
840     parser.add_argument("--lamda", nargs='+', type=float, default=[0.0008, 0.0004,
841                                     0.0016])
842     parser.add_argument("--th1", nargs='+', type=int, default=[1, 4, 5])
843     parser.add_argument("--th2", nargs='+', type=int, default=[2, 5, 7])
844     parser.add_argument("--NL", type=int, default=3)
845     parser.add_argument("--concats", type=tuple, default=(1, 0, 1))
846     parser.add_argument("--act", nargs='+', type=str, default=["triangle", "triangle", "
            relu"])
847     parser.add_argument("--alleps", nargs='+', type=int, default=[6, 6, 13])
848     parser.add_argument("--device_num", type=int, default=0)
849     parser.add_argument("--seed_num", type=int, default=1234)
850     parser.add_argument("--save_model", action="store_true")
851     return parser
852
853 if __name__ == "__main__":
854     parser = argparse.ArgumentParser('SCFF Split Learning', parents=[get_arguments()])
855     args = parser.parse_args()

```

```

854
855 for arg in vars(args):
856     print(f"{arg} = {getattr(args, arg)}")
857
858 NL = args.NL
859 assert NL >= 2, f"Split learning requires NL >= 2, got {NL}"
860
861 device = 'cuda:' + str(args.device_num) if torch.cuda.is_available() else 'cpu'
862 print(f"Using device: {device}")
863
864 torch.manual_seed(args.seed_num)
865
866 # Load config
867 with open('config.json', 'r') as f:
868     config_json = json.load(f)
869
870 layer_configs = config_json['CIFAR']['layer_configs'][:NL]
871 opt_configs = config_json['CIFAR']['opt_configs'][:NL]
872 concats = args.concats
873 act_list = args.act
874
875 # Load data
876 loaders = get_train(batchsize=100, augment="no", Factor=1)
877
878 # ---- Build Client (Layer 0) ----
879 lc0 = layer_configs[0]
880 oc0 = opt_configs[0]
881 net0, pool0, extra_pool0, _, _ = create_layer(lc0, oc0, load_params=False, device=
    device, act=act_list[0])
882 net0.concat = bool(concats[0])
883 opt0 = AdamW(net0.parameters(), lr=args.lr[0], weight_decay=args.weight_decay[0])
884 sched0 = ExponentialLR(opt0, args.gamma[0])
885
886 client = SplitClient(net0, pool0, extra_pool0, opt0, sched0, device)
887
888 # ---- Build Server (Layers 1..NL-1) ----
889 server_nets = []
890 server_pools = []
891 server_extra_pools = []
892 server_optimizers = []
893 server_schedulers = []
894
895 for i in range(1, NL):
896     lc = layer_configs[i]
897     oc = opt_configs[i]
898     net_i, pool_i, extra_pool_i, _, _ = create_layer(lc, oc, load_params=False,
        device=device, act=act_list[i])
899     net_i.concat = bool(concats[i])
900     opt_i = AdamW(net_i.parameters(), lr=args.lr[i], weight_decay=args.weight_decay[
        i])
901     sched_i = ExponentialLR(opt_i, args.gamma[i])
902
903     server_nets.append(net_i)
904     server_pools.append(pool_i)
905     server_extra_pools.append(extra_pool_i)
906     server_optimizers.append(opt_i)
907     server_schedulers.append(sched_i)
908
909 server = SplitServer(server_nets, server_pools, server_extra_pools,
910     server_optimizers, server_schedulers, device)
911
912 # ---- Evaluation config ----
913 eval_config = EvaluationConfig(
914     device=device,
915     dims=(1, 2, 3),
916     dims_in=(1, 2, 3),
917     dims_out=(1, 2, 3),
918     stdnorm_out=True,

```

```

919         out_dropout=0.2,
920         Layer_out=[2, 1, 0],
921         pre_std=True,
922         all_neurons=False
923     )
924
925     # ---- Pre-training validation ----
926     run_validation_checks(client, server, device, loaders[0], config_json)
927
928     # ---- Build trainer ----
929     trainer = SplitTrainer(
930         client=client,
931         server=server,
932         loaders=loaders,
933         eval_config=eval_config,
934         threshold1=args.th1,
935         threshold2=args.th2,
936         lamda=args.lamda,
937         period=args.period,
938         alleps=args.alleps,
939         p=1,
940         a=1,
941         b=1
942     )
943
944     # ---- Train ----
945     total_epochs = max(args.alleps)
946     Dims = trainer.train(total_epochs)
947     print(f"Layer output dims: {Dims}")
948
949     # ---- Evaluate ----
950     acc_train, acc_test = trainer.evaluate()
951     print(f"\nFinal Results:")
952     print(f"  Train accuracy: {100 * acc_train:.2f}%")
953     print(f"  Test accuracy: {100 * acc_test:.2f}%")
954
955     # ---- Save model if requested ----
956     if args.save_model:
957         import os
958         os.makedirs('./results', exist_ok=True)
959         torch.save(client.net.state_dict(), './results/params_CIFAR_split_client_10.pth')
960
961         for i, net in enumerate(server.nets):
962             torch.save(net.state_dict(), f'./results/params_CIFAR_split_server_1{i + 1}.pth')
963         print("Models saved.")

```

Listing 2: SCFF_CIFAR_Split.py — single-client split learning implementation. Layer 0 on client, Layers 1–2 on server. Includes SplitClient, SplitServer, SplitTrainer, and pre-training validation.

A.3 SCFF_CIFAR_MultiClient.py — Multi-Client Sequential Relay

```

1  """
2  SCFF CIFAR-10 Multi-Client Split Learning
3  =====
4  1 Server (Layers 1-2) + N Clients (each holds Layer 0 copy + data shard).
5
6  Sequential/Relay approach:
7  - CIFAR-10 is split equally among N clients
8  - Each round, clients take turns:
9      Client_i trains Layer 0 on its shard → sends activations to server
10 - Between clients, Layer 0 weights are relayed (passed to next client)
11 - Server layers are shared and train on activations from ALL clients

```

```

12 - After cycling through all N clients = 1 round  $\approx$  1 epoch over full dataset
13 - Multiple rounds  $\rightarrow$  accuracy within  $\pm 1-2\%$  of baseline
14
15 Usage:
16 python SCFF_CIFAR_MultiClient.py --num_clients 10
17 python SCFF_CIFAR_MultiClient.py --num_clients 5
18 python SCFF_CIFAR_MultiClient.py --num_clients 2
19 """
20
21 import os
22 os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
23
24 import torch
25 import torch.nn as nn
26 import torch.optim as optim
27 from torch.optim import AdamW
28 from torch.optim.lr_scheduler import ExponentialLR, StepLR
29
30 import torch.nn.functional as F
31 import torch.nn.init as init
32 import matplotlib
33 matplotlib.use('Agg')
34 import matplotlib.pyplot as plt
35
36 import torchvision
37 from torchvision.transforms import transforms, ToPILImage
38 from torch.utils.data import DataLoader, Subset
39 import argparse
40 import copy
41 import math
42 import json
43 import numpy as np
44
45 # =====
46 # Reproducibility
47 # =====
48 torch.manual_seed(1234)
49
50 # =====
51 # Transforms
52 # =====
53 s = 0.5
54 transform1 = transforms.Compose([
55     transforms.RandomResizedCrop(size=(32, 32), scale=(0.8, 1.0), ratio=(0.75, 1.33)),
56     transforms.RandomHorizontalFlip(),
57     transforms.RandomApply([transforms.ColorJitter(brightness=0.8*s, contrast=0.8*s,
58         saturation=0.8*s, hue=0.2*s)], p=0.8),
59     transforms.ToTensor(),
60     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
61 ])
62
63 transform_train = transforms.Compose([
64     transforms.RandomHorizontalFlip(),
65     transforms.ToTensor(),
66     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
67 ])
68
69 transform_test = transforms.Compose([
70     transforms.ToTensor(),
71     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
72 ])
73
74 # =====
75 # Dataset classes
76 # =====
77 class DualAugmentCIFAR10(torchvision.datasets.CIFAR10):
78     def __init__(self, root, augment="No", *args, **kwargs):

```

```

79     super().__init__(root, *args, **kwargs)
80     self.augment = augment
81
82     def __getitem__(self, index):
83         img, target = self.data[index], self.targets[index]
84         img_pil = ToPILImage()(img)
85         img_original = transform_train(img_pil)
86         if self.augment == "single":
87             img1 = transform1(img_pil)
88             return img_original, img1, img_original, target
89         elif self.augment == "dual":
90             img1 = transform1(img_pil)
91             img2 = transforms.Compose([
92                 transforms.RandomResizedCrop(size=(32, 32), scale=(0.8, 1.0)),
93                 transforms.RandomHorizontalFlip(),
94                 transforms.RandomApply([transforms.ColorJitter(0.4, 0.4, 0.4, 0.1)], p
95                                         =0.8),
96                 transforms.ToTensor(),
97                 transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))
98             ])(img_pil)
99             return img_original, img1, img2, target
100         else:
101             return img_original, target
102
103 class DualAugmentCIFAR10_test(torchvision.datasets.CIFAR10):
104     def __init__(self, aug=False, *args, **kwargs):
105         super().__init__(*args, **kwargs)
106         self.aug = aug
107
108     def __getitem__(self, index):
109         img, target = self.data[index], self.targets[index]
110         img = ToPILImage()(img)
111         img = transform_train(img) if self.aug else transform_test(img)
112         return img, target
113
114
115 # =====
116 # Data splitting for multi-client
117 # =====
118 def get_client_loaders(num_clients, batchsize, iid=True):
119     """
120     Split CIFAR-10 training set into 'num_clients' equal shards.
121
122     Args:
123         num_clients: number of clients
124         batchsize: batch size per client
125         iid: if True, random IID split; if False, sort by class (non-IID)
126
127     Returns:
128         client_trainloaders: list of DataLoaders, one per client
129         sup_trainloader: full supervised train loader (for evaluation)
130         testloader: test loader
131     """
132     torch.manual_seed(1234)
133
134     trainset = DualAugmentCIFAR10(root='./data', train=True, download=True, augment="no")
135     sup_trainset = DualAugmentCIFAR10_test(root='./data', aug=True, train=True, download=True)
136     testset = DualAugmentCIFAR10_test(root='./data', aug=False, train=False, download=True)
137
138     n_total = len(trainset) # 50,000
139     shard_size = n_total // num_clients
140
141     if iid:

```

```

142     # Random IID split
143     indices = torch.randperm(n_total).tolist()
144 else:
145     # Non-IID: sort by label (harder scenario)
146     targets = torch.tensor(trainset.targets)
147     indices = targets.argsort().tolist()
148
149 client_trainloaders = []
150 for c in range(num_clients):
151     start = c * shard_size
152     end = start + shard_size if c < num_clients - 1 else n_total
153     client_indices = indices[start:end]
154     client_subset = Subset(trainset, client_indices)
155     loader = DataLoader(client_subset, batch_size=batchsize, shuffle=True,
156                        num_workers=2)
157     client_trainloaders.append(loader)
158
159 print(f>Data split: {n_total} samples → {num_clients} clients × {shard_size} samples
160      each")
161
162 sup_trainloader = DataLoader(
163     Subset(sup_trainset, list(range(n_total))),
164     batch_size=64, shuffle=True
165 )
166 testloader = DataLoader(testset, batch_size=1000, shuffle=False, num_workers=2)
167
168 return client_trainloaders, sup_trainloader, testloader
169
170 # =====
171 # Utility functions (same as baseline)
172 # =====
173 def get_pos_neg_batch_imgcats(batch_pos1, batch_pos2, p=1):
174     batch_size = len(batch_pos1)
175     batch_pos = torch.cat((batch_pos1, batch_pos2), dim=1)
176     random_indices = (torch.randperm(batch_size - 1) + 1)[:min(p, batch_size - 1)]
177     labels = torch.arange(batch_size)
178     batch_negs = []
179     for i in random_indices:
180         batch_neg = batch_pos2[(labels + i) % batch_size]
181         batch_neg = torch.cat((batch_pos1, batch_neg), dim=1)
182         batch_negs.append(batch_neg)
183     return batch_pos, torch.cat(batch_negs)
184
185 def stdnorm(x, dims=[1, 2, 3]):
186     x = x - torch.mean(x, dim=dims, keepdim=True)
187     x = x / (1e-10 + torch.std(x, dim=dims, keepdim=True))
188     return x
189
190
191 class standardnorm(nn.Module):
192     def __init__(self, dims=[1, 2, 3]):
193         super().__init__()
194         self.dims = dims
195
196     def forward(self, x):
197         x = x - torch.mean(x, dim=self.dims, keepdim=True)
198         x = x / (1e-10 + torch.std(x, dim=self.dims, keepdim=True))
199         return x
200
201
202 class L2norm(nn.Module):
203     def __init__(self, dims=[1, 2, 3]):
204         super().__init__()
205         self.dims = dims
206
207     def forward(self, x):

```

```

208         return x / (x.norm(p=2, dim=self.dims, keepdim=True) + 1e-10)
209
210
211 class triangle(nn.Module):
212     def __init__(self):
213         super().__init__()
214
215     def forward(self, x):
216         x = x - torch.mean(x, axis=1, keepdims=True)
217         return F.relu(x)
218
219
220 # =====
221 # Conv2d layer (same as baseline)
222 # =====
223 class Conv2d(nn.Module):
224     def __init__(self, input_channels, output_channels, kernel_size, pad=0,
225                 batchnorm=False, normdims=[1, 2, 3], norm="stdnorm", bias=True,
226                 dropout=0.0, padding_mode="reflect", concat=True, act="relu"):
227         super().__init__()
228         self.input_channels = input_channels
229         self.output_channels = output_channels
230         self.kernel_size = kernel_size
231         self.normdims = normdims
232         self.concat = concat
233         self.relu = torch.nn.ReLU()
234         self.conv_layer = nn.Conv2d(in_channels=input_channels, out_channels=
            output_channels,
235                                     kernel_size=kernel_size, bias=bias)
236         init.xavier_uniform_(self.conv_layer.weight)
237         self.padding_mode = padding_mode
238         self.F_padding = (pad, pad, pad, pad)
239         self.act = torch.nn.ReLU() if act == 'relu' else triangle()
240         if batchnorm:
241             self.bn1 = nn.BatchNorm2d(self.input_channels, affine=False)
242         else:
243             self.bn1 = nn.Identity()
244         if norm == "L2norm":
245             self.norm = L2norm(dims=normdims)
246         elif norm == "stdnorm":
247             self.norm = standardnorm(dims=normdims)
248         else:
249             self.norm = nn.Identity()
250
251     def forward(self, x):
252         x = self.bn1(x)
253         x = F.pad(x, self.F_padding, self.padding_mode)
254         x = self.norm(x)
255         if self.concat:
256             lenchannel = x.size(1) // 2
257             out = self.conv_layer(x[:, :lenchannel]) + self.conv_layer(x[:, lenchannel
                :])
258         else:
259             out = self.conv_layer(x)
260         return out
261
262
263 # =====
264 # Helpers
265 # =====
266 class CustomStepLR(StepLR):
267     def __init__(self, optimizer, nb_epochs):
268         threshold_ratios = [0.2, 0.35, 0.5, 0.6, 0.7, 0.8, 0.9]
269         self.step_thresold = [int(nb_epochs * r) for r in threshold_ratios]
270         super().__init__(optimizer, -1, False)
271
272     def get_lr(self):
273         if self.last_epoch in self.step_thresold:

```

```

274         return [group['lr'] * 0.5 for group in self.optimizer.param_groups]
275     return [group['lr'] for group in self.optimizer.param_groups]
276
277
278 class EvaluationConfig:
279     def __init__(self, device, dims, dims_in, dims_out, stdnorm_out, out_dropout,
280                 Layer_out, pre_std, all_neurons):
281         self.device = device
282         self.dims = dims
283         self.dims_in = dims_in
284         self.dims_out = dims_out
285         self.stdnorm_out = stdnorm_out
286         self.out_dropout = out_dropout
287         self.Layer_out = Layer_out
288         self.all_neurons = all_neurons
289         self.pre_std = pre_std
290
291
292 def calculate_output_length(dims, nets, extra_pool, Layer, all_neurons):
293     lengths = 0
294     if all_neurons:
295         for i, length in enumerate(dims):
296             if i in Layer:
297                 lengths += length
298     else:
299         for i, length in enumerate(dims):
300             if i in Layer:
301                 len_after_pool = math.ceil(
302                     (math.sqrt(length / nets[i].output_channels) - extra_pool[i].
303                      kernel_size)
304                     / extra_pool[i].stride + 1)
305                 lengths += len_after_pool * len_after_pool * nets[i].output_channels
306
307     return lengths
308
309 def build_classifier(lengths, config):
310     classifier = nn.Sequential(
311         nn.Dropout(config.out_dropout),
312         nn.Linear(lengths, 10)
313     ).to(config.device)
314     if torch.cuda.device_count() > 2:
315         classifier = nn.DataParallel(classifier)
316     return classifier
317
318 def create_layer(layer_config, opt_config, device, act):
319     net = Conv2d(layer_config["ch_in"], layer_config["channels"],
320                 (layer_config["kernel_size"], layer_config["kernel_size"]),
321                 pad=layer_config["pad"], norm="stdnorm",
322                 padding_mode=layer_config["padding_mode"], act=act)
323     if layer_config["pooltype"] == 'Avg':
324         pool = nn.AvgPool2d(kernel_size=layer_config["pool_size"],
325                             stride=layer_config["stride_size"],
326                             padding=layer_config["padding"], ceil_mode=True)
327     else:
328         pool = nn.MaxPool2d(kernel_size=layer_config["pool_size"],
329                             stride=layer_config["stride_size"],
330                             padding=layer_config["padding"], ceil_mode=True)
331     extra_pool = nn.AvgPool2d(kernel_size=layer_config["extra_pool_size"],
332                               stride=layer_config["extra_pool_size"],
333                               padding=0, ceil_mode=True)
334     net.to(device)
335     return net, pool, extra_pool
336
337
338 # =====
339 # SplitClient: Holds Layer 0 weights (relayed between clients)
340 # =====

```

```

341 class SplitClient:
342     """
343     Represents the client-side model (Layer 0).
344     In multi-client mode, there's ONE set of Layer 0 weights
345     that gets relayed from client to client.
346     """
347
348     def __init__(self, net, pool, extra_pool, device, lr, weight_decay, gamma):
349         self.net = net
350         self.pool = pool
351         self.extra_pool = extra_pool
352         self.device = device
353         # Store optimizer hyperparams for re-creation after weight relay
354         self.lr = lr
355         self.weight_decay = weight_decay
356         self.gamma = gamma
357         # Create optimizer/scheduler
358         self.optimizer = AdamW(net.parameters(), lr=lr, weight_decay=weight_decay)
359         self.scheduler = ExponentialLR(self.optimizer, gamma)
360
361     def get_weights(self):
362         """Get Layer 0 weights + optimizer state for relay."""
363         return {
364             'model': copy.deepcopy(self.net.state_dict()),
365             'optimizer': copy.deepcopy(self.optimizer.state_dict()),
366         }
367
368     def set_weights(self, state):
369         """Receive relayed weights from previous client."""
370         self.net.load_state_dict(state['model'])
371         # Recreate optimizer to bind to current net params, then load state
372         self.optimizer = AdamW(self.net.parameters(), lr=self.lr, weight_decay=self.
            weight_decay)
373         self.optimizer.load_state_dict(state['optimizer'])
374         # Update scheduler to track new optimizer
375         self.scheduler = ExponentialLR(self.optimizer, self.gamma)
376
377     def train_step(self, x, threshold1, threshold2, a, b, lamda, p=1):
378         """Train Layer 0 on a batch. Returns pooled activations for server."""
379         self.net.train()
380         x = stdnorm(x, dims=[1, 2, 3])
381         x_pos, x_neg = get_pos_neg_batch_imgcats(x, x, p=p)
382
383         out_pos = self.net(x_pos)
384         out_neg = self.net(x_neg)
385
386         goodness_pos = self.net.relu(out_pos).pow(2).mean([1])
387         goodness_neg = self.net.relu(out_neg).pow(2).mean([1])
388
389         self.optimizer.zero_grad()
390         loss = (torch.log(1 + torch.exp(a * (-goodness_pos + threshold1))).mean([1, 2]).
            mean()
            + torch.log(1 + torch.exp(b * (goodness_neg - threshold2))).mean([1, 2]).
            mean()
            + lamda * torch.norm(goodness_pos, p=2, dim=(1, 2)).mean())
391
392         loss.backward()
393         self.optimizer.step()
394
395         x_pos_pooled = self.pool(self.net.act(out_pos)).detach()
396         x_neg_pooled = self.pool(self.net.act(out_neg)).detach()
397
398         gp = torch.mean(goodness_pos.mean([1, 2])).item()
399         gn = torch.mean(goodness_neg.mean([1, 2])).item()
400
401         return x_pos_pooled, x_neg_pooled, gp, gn
402
403
404     def extract_features(self, x, dims_in, dims_out, stdnorm_out, Layer_out):
405         """Inference: extract Layer 0 features."""

```

```

406         self.net.eval()
407         outputs = []
408         with torch.no_grad():
409             x = stdnorm(x, dims=dims_in)
410             x = torch.cat((x, x), dim=1)
411             x = self.pool(self.net.act(self.net(x)))
412             out = self.extra_pool(x)
413             if stdnorm_out:
414                 out = stdnorm(out, dims=dims_out)
415                 out = out.flatten(start_dim=1)
416             if 0 in Layer_out:
417                 outputs.append(out)
418         return x, outputs
419
420
421 # =====
422 # SplitServer: Holds Layers 1-2 (shared across all clients)
423 # =====
424 class SplitServer:
425     """Server holds Layers 1..NL-1, shared by all clients."""
426
427     def __init__(self, nets, pools, extra_pools, optimizers, schedulers, device):
428         self.nets = nets
429         self.pools = pools
430         self.extra_pools = extra_pools
431         self.optimizers = optimizers
432         self.schedulers = schedulers
433         self.device = device
434
435     def train_step(self, x, x_neg, threshold1_list, threshold2_list, a, b, lamda_list,
436                   alleps, epoch, p=1):
437         """Train server layers on received activations."""
438         gp, gn = 0, 0
439         for i, net in enumerate(self.nets):
440             net.train()
441             if net.concat:
442                 x = stdnorm(x, dims=[1, 2, 3])
443                 x, x_neg = get_pos_neg_batch_imgcats(x, x, p=p)
444
445             out_pos = net(x)
446             out_neg = net(x_neg)
447
448             goodness_pos = net.relu(out_pos).pow(2).mean([1])
449             goodness_neg = net.relu(out_neg).pow(2).mean([1])
450
451             if epoch < alleps[i]:
452                 self.optimizers[i].zero_grad()
453                 loss = (torch.log(1 + torch.exp(a * (-goodness_pos + threshold1_list[i]
454                 )))
455                        + torch.log(1 + torch.exp(b * (goodness_neg - threshold2_list[i]
456                        )))
457                        + lamda_list[i] * torch.norm(goodness_pos, p=2, dim=(1, 2)).mean(
458                        ))
459                 loss.backward()
460                 self.optimizers[i].step()
461
462                 x = self.pools[i](net.act(out_pos)).detach()
463                 x_neg = self.pools[i](net.act(out_neg)).detach()
464
465                 gp = torch.mean(goodness_pos.mean([1, 2])).item()
466                 gn = torch.mean(goodness_neg.mean([1, 2])).item()
467
468         return gp, gn
469
470     def extract_features(self, x, dims_in, dims_out, stdnorm_out, Layer_out):
471         """Inference: extract features from server layers."""
472         outputs = []
473         for i, net in enumerate(self.nets):

```

```

471         net.eval()
472         with torch.no_grad():
473             global_layer_idx = i + 1
474             if net.concat:
475                 x = stdnorm(x, dims=dims_in)
476                 x = torch.cat((x, x), dim=1)
477             x = self.pools[i](net.act(net(x)))
478             out = self.extra_pools[i](x)
479             if stdnorm_out:
480                 out = stdnorm(out, dims=dims_out)
481             out = out.flatten(start_dim=1)
482             if global_layer_idx in Layer_out:
483                 outputs.append(out)
484         return outputs
485
486
487 # =====
488 # MultiClientTrainer: Orchestrates N clients + 1 server
489 # =====
490 class MultiClientTrainer:
491     """
492     Sequential/Relay Split Learning with N clients.
493
494     Each "round":
495         for client_id in 0..N-1:
496             1. Client receives relayed Layer 0 weights
497             2. Client trains Layer 0 on its local data shard
498             3. Client sends activations to server
499             4. Server trains Layers 1-2 on those activations
500             5. Client passes Layer 0 weights to next client
501
502     After N clients = 1 round  $\approx$  1 full epoch over 50k samples.
503     """
504
505     def __init__(self, client, server, client_loaders, eval_loaders, eval_config,
506                  threshold1, threshold2, lamda, period, alleps,
507                  p=1, a=1, b=1, num_clients=10):
508         self.client = client
509         self.server = server
510         self.client_loaders = client_loaders # list of N DataLoaders
511         self.eval_loaders = eval_loaders # (sup_trainloader, testloader)
512         self.eval_config = eval_config
513         self.threshold1 = threshold1
514         self.threshold2 = threshold2
515         self.lamda = lamda
516         self.period = period
517         self.alleps = alleps
518         self.p = p
519         self.a = a
520         self.b = b
521         self.num_clients = num_clients
522         self.Dims = []
523
524     def train(self, total_rounds):
525         """
526         Run multi-client training.
527
528         Args:
529             total_rounds: number of full rounds (each round = all N clients train once)
530                         Equivalent to epochs in the single-client case.
531         """
532         device = self.eval_config.device
533         firstpass = True
534         nbbatches = 0
535
536         for round_num in range(total_rounds):
537             print(f"\n{'='*60}")
538             print(f"Round {round_num}/{total_rounds} ")

```

```

539         f"(equivalent epoch over full dataset)")
540     print(f"{'='*60}")
541
542     round_gp_accum = 0
543     round_gn_accum = 0
544     round_batch_count = 0
545
546     # --- Sequential relay through all clients ---
547     for client_id in range(self.num_clients):
548         client_loader = self.client_loaders[client_id]
549
550         self.client.net.train()
551         for net in self.server.nets:
552             net.train()
553
554         client_gp = 0
555         client_gn = 0
556
557         for numbatch, (x, _) in enumerate(client_loader):
558             nbbatches += 1
559             x = x.to(device)
560
561             # --- Client: Layer 0 ---
562             x_pos_pooled, x_neg_pooled, gp_c, gn_c = self.client.train_step(
563                 x, self.threshold1[0], self.threshold2[0],
564                 self.a, self.b, self.lamda[0], p=self.p
565             )
566
567             if firstpass:
568                 _, ch, h, w = x_pos_pooled.shape
569                 self.Dims.append(ch * h * w)
570                 print(f"Layer 0 : shape {x_pos_pooled.shape}")
571
572             # Client LR scheduler
573             if round_num < self.alleps[0] and (nbbatches + 1) % self.period[0]
574                 == 0:
575                 self.client.scheduler.step()
576
577             # --- Server: Layers 1+ ---
578             server_th1 = self.threshold1[1:]
579             server_th2 = self.threshold2[1:]
580             server_lamda = self.lamda[1:]
581             server_alleps = self.alleps[1:]
582
583             gp_s, gn_s = self.server.train_step(
584                 x_pos_pooled, x_neg_pooled,
585                 server_th1, server_th2,
586                 self.a, self.b, server_lamda,
587                 server_alleps, round_num, p=self.p
588             )
589
590             if firstpass:
591                 dummy_x = x_pos_pooled
592                 for si, snet in enumerate(self.server.nets):
593                     with torch.no_grad():
594                         if snet.concat:
595                             dummy_x = stdnorm(dummy_x, dims=[1, 2, 3])
596                             dummy_x = torch.cat((dummy_x, dummy_x), dim=1)
597                             dummy_x = self.server.pools[si](snet.act(snet(dummy_x)))
598                             _, ch, h, w = dummy_x.shape
599                             self.Dims.append(ch * h * w)
600                             print(f"Layer {si + 1} : shape {dummy_x.shape}")
601                 firstpass = False
602
603             # Server LR schedulers
604             for si in range(len(self.server.nets)):
605                 if round_num < self.alleps[si + 1] and (nbbatches + 1) % self.
606                     period[si + 1] == 0:

```

```

605         self.server.schedulers[si].step()
606
607         client_gp += gp_s
608         client_gn += gn_s
609         round_batch_count += 1
610
611         n_batches_client = len(client_loader)
612         print(f" Client {client_id}: {n_batches_client} batches, "
613               f"pos_goodness={client_gp/max(n_batches_client,1):.4f}, "
614               f"neg_goodness={client_gn/max(n_batches_client,1):.4f}")
615
616         round_gp_accum += client_gp
617         round_gn_accum += client_gn
618
619         # End of round summary
620         print(f"Round {round_num} done: "
621               f"avg_pos={round_gp_accum/max(round_batch_count,1):.4f}, "
622               f"avg_neg={round_gn_accum/max(round_batch_count,1):.4f}")
623
624     print("\nTraining complete.")
625     return self.Dims
626
627 def evaluate(self):
628     """Evaluate by training a linear readout on features from all layers."""
629     config = self.eval_config
630     sup_trainloader, testloader = self.eval_loaders
631
632     current_rng_state = torch.get_rng_state()
633     torch.manual_seed(42)
634
635     all_nets = [self.client.net] + self.server.nets
636     all_extra_pools = [self.client.extra_pool] + self.server.extra_pools
637
638     lengths = calculate_output_length(self.Dims, all_nets, all_extra_pools,
639                                     config.Layer_out, config.all_neurons)
640     print(f"Classifier input length: {lengths}")
641     classifier = build_classifier(lengths, config)
642
643     optimizer = optim.Adam(classifier.parameters(), lr=0.001)
644     lr_scheduler = CustomStepLR(optimizer, nb_epochs=50)
645     criterion = nn.CrossEntropyLoss()
646
647     self.client.net.eval()
648     for net in self.server.nets:
649         net.eval()
650
651     train_accs = []
652     test_accs = []
653     last_test_acc = 0
654
655     for epoch_eval in range(50):
656         classifier.train()
657         correct = 0
658         total = 0
659         for x, labels in sup_trainloader:
660             x = x.to(config.device)
661             labels = labels.to(config.device)
662
663             with torch.no_grad():
664                 x_after_client, client_outputs = self.client.extract_features(
665                     x, config.dims_in, config.dims_out, config.stdnorm_out, config.
666                         Layer_out)
667                 server_outputs = self.server.extract_features(
668                     x_after_client, config.dims_in, config.dims_out, config.
669                         stdnorm_out, config.Layer_out)
670
671             outputs = client_outputs + server_outputs
672             outputs = torch.cat(outputs, dim=1)

```

```

671         optimizer.zero_grad()
672         preds = classifier(outputs)
673         loss = criterion(preds, labels)
674         loss.backward()
675         optimizer.step()
676
677         _, predicted = torch.max(preds.data, 1)
678         total += labels.size(0)
679         correct += (predicted == labels).sum().item()
680
681     train_acc = correct / total
682     lr_scheduler.step()
683
684     if epoch_eval % 20 == 0 or epoch_eval == 49:
685         print(f'Readout epoch {epoch_eval}: train acc = {100 * train_acc:.2f}%')
686         test_acc = self._test_readout(classifier, testloader, criterion, config,
687                                     epoch_eval)
688         last_test_acc = test_acc
689
690     train_accs.append(train_acc)
691     test_accs.append(last_test_acc)
692
693     final_train = train_acc
694     final_test = self._test_readout(classifier, testloader, criterion, config, 49)
695
696     torch.set_rng_state(current_rng_state)
697     self._plot_accuracy(train_accs, test_accs)
698
699     return final_train, final_test
700
701 def _test_readout(self, classifier, loader, criterion, config, epoch):
702     classifier.eval()
703     correct = 0
704     total = 0
705     with torch.no_grad():
706         for x, labels in loader:
707             x = x.to(config.device)
708             labels = labels.to(config.device)
709
710             x_after_client, client_outputs = self.client.extract_features(
711                 x, config.dims_in, config.dims_out, config.stdnorm_out, config.
712                 Layer_out)
713             server_outputs = self.server.extract_features(
714                 x_after_client, config.dims_in, config.dims_out, config.stdnorm_out,
715                 config.Layer_out)
716
717             outputs = client_outputs + server_outputs
718             outputs = torch.cat(outputs, dim=1)
719             preds = classifier(outputs)
720
721             _, predicted = torch.max(preds.data, 1)
722             total += labels.size(0)
723             correct += (predicted == labels).sum().item()
724
725     acc = correct / total
726     print(f' Test accuracy: {100 * acc:.2f}%')
727     classifier.train()
728     return acc
729
730 def _plot_accuracy(self, train_accs, test_accs):
731     plt.figure(figsize=(10, 6))
732     epochs = list(range(1, len(train_accs) + 1))
733     plt.plot(epochs, [a * 100 for a in train_accs], label='Train Accuracy', marker='
734     .')
735     plt.plot(epochs, [a * 100 for a in test_accs], label='Test Accuracy', marker='
736     .')
737     plt.xlabel('Readout Epoch')

```

```

734     plt.ylabel('Accuracy (%)')
735     plt.title(f'SCFF Multi-Client Split Learning ({self.num_clients} clients) -
        Readout Accuracy')
736     plt.legend()
737     plt.grid(True)
738     plt.tight_layout()
739     outpath = f'/home/claw/.openclaw/workspace/scff_split/multiclient_{self.
        num_clients}_accuracy.png'
740     plt.savefig(outpath, dpi=150)
741     plt.close()
742     print(f"Plot saved to {outpath}")
743
744
745 # =====
746 # Main
747 # =====
748 def get_arguments():
749     parser = argparse.ArgumentParser(description="SCFF Multi-Client Split Learning",
        add_help=False)
750     parser.add_argument("--num_clients", type=int, default=10, help="Number of clients (
        data split equally)")
751     parser.add_argument("--iid", action="store_true", default=True, help="IID data split
        (default: True)")
752     parser.add_argument("--non_iid", action="store_true", help="Non-IID data split (
        sorted by class)")
753     parser.add_argument("--lr", nargs='+', type=float, default=[0.02, 0.001, 0.0004])
754     parser.add_argument("--gamma", nargs='+', type=float, default=[0.99, 0.9, 0.99])
755     parser.add_argument("--period", nargs='+', type=int, default=[500, 500, 500])
756     parser.add_argument("--weight_decay", nargs='+', type=float, default=[0.0001,
        0.0003, 0.0001])
757     parser.add_argument("--lamda", nargs='+', type=float, default=[0.0008, 0.0004,
        0.0016])
758     parser.add_argument("--th1", nargs='+', type=int, default=[1, 4, 5])
759     parser.add_argument("--th2", nargs='+', type=int, default=[2, 5, 7])
760     parser.add_argument("--NL", type=int, default=3)
761     parser.add_argument("--concat", type=tuple, default=(1, 0, 1))
762     parser.add_argument("--act", nargs='+', type=str, default=["triangle", "triangle", "
        relu"])
763     parser.add_argument("--alleps", nargs='+', type=int, default=[6, 6, 13])
764     parser.add_argument("--device_num", type=int, default=0)
765     parser.add_argument("--seed_num", type=int, default=1234)
766     parser.add_argument("--save_model", action="store_true")
767     return parser
768
769
770 if __name__ == "__main__":
771     parser = argparse.ArgumentParser('SCFF Multi-Client', parents=[get_arguments()])
772     args = parser.parse_args()
773
774     for arg in vars(args):
775         print(f"{arg} = {getattr(args, arg)}")
776
777     NL = args.NL
778     assert NL >= 2
779
780     device = 'cuda:' + str(args.device_num) if torch.cuda.is_available() else 'cpu'
781     print(f"Using device: {device}")
782     torch.manual_seed(args.seed_num)
783
784     with open('config.json', 'r') as f:
785         config_json = json.load(f)
786
787     layer_configs = config_json['CIFAR']['layer_configs'][:NL]
788     opt_configs = config_json['CIFAR']['opt_configs'][:NL]
789     concat = args.concat
790     act_list = args.act
791
792     iid = not args.non_iid

```

```

793 num_clients = args.num_clients
794
795 # --- Load data (split among clients) ---
796 client_loaders, sup_trainloader, testloader = get_client_loaders(
797     num_clients=num_clients, batchsize=100, iid=iid
798 )
799
800 # --- Build Client (Layer 0) - single set of weights relayed ---
801 lc0 = layer_configs[0]
802 net0, pool0, extra_pool0 = create_layer(lc0, opt_configs[0], device, act_list[0])
803 net0.concat = bool(concat0[0])
804
805 client = SplitClient(
806     net0, pool0, extra_pool0, device,
807     lr=args.lr[0], weight_decay=args.weight_decay[0], gamma=args.gamma[0]
808 )
809
810 # --- Build Server (Layers 1..NL-1) ---
811 server_nets = []
812 server_pools = []
813 server_extra_pools = []
814 server_optimizers = []
815 server_schedulers = []
816
817 for i in range(1, NL):
818     lc = layer_configs[i]
819     net_i, pool_i, extra_pool_i = create_layer(lc, opt_configs[i], device, act_list[
820         i])
821     net_i.concat = bool(concat0[i])
822     opt_i = AdamW(net_i.parameters(), lr=args.lr[i], weight_decay=args.weight_decay[
823         i])
824     sched_i = ExponentialLR(opt_i, args.gamma[i])
825
826     server_nets.append(net_i)
827     server_pools.append(pool_i)
828     server_extra_pools.append(extra_pool_i)
829     server_optimizers.append(opt_i)
830     server_schedulers.append(sched_i)
831
832 server = SplitServer(server_nets, server_pools, server_extra_pools,
833     server_optimizers, server_schedulers, device)
834
835 # --- Evaluation config ---
836 eval_config = EvaluationConfig(
837     device=device, dims=(1, 2, 3), dims_in=(1, 2, 3), dims_out=(1, 2, 3),
838     stdnorm_out=True, out_dropout=0.2, Layer_out=[2, 1, 0],
839     pre_std=True, all_neurons=False
840 )
841
842 # --- Build trainer ---
843 trainer = MultiClientTrainer(
844     client=client,
845     server=server,
846     client_loaders=client_loaders,
847     eval_loaders=(sup_trainloader, testloader),
848     eval_config=eval_config,
849     threshold1=args.th1,
850     threshold2=args.th2,
851     lamda=args.lamda,
852     period=args.period,
853     alleps=args.alleps,
854     p=1, a=1, b=1,
855     num_clients=num_clients
856 )
857
858 # --- Train ---
859 total_rounds = max(args.alleps) # each round = full pass through all clients
860 Dims = trainer.train(total_rounds)

```

```

859     print(f"Layer output dims: {Dims}")
860
861     # --- Evaluate ---
862     acc_train, acc_test = trainer.evaluate()
863     print(f"\n{'='*60}")
864     print(f"FINAL RESULTS ({num_clients} clients, {'IID' if iid else 'Non-IID'} split)")
865     print(f"{'='*60}")
866     print(f"    Train accuracy: {100 * acc_train:.2f}%")
867     print(f"    Test accuracy:   {100 * acc_test:.2f}%")
868
869     # --- Save ---
870     if args.save_model:
871         os.makedirs('./results', exist_ok=True)
872         torch.save(client.net.state_dict(), './results/params_CIFAR_multiclient_l0.pth')
873         for i, net in enumerate(server.nets):
874             torch.save(net.state_dict(), f'./results/params_CIFAR_multiclient_l{i + 1}.pth')
875     print("Models saved.")

```

Listing 3: SCFF_CIFAR_MultiClient.py — multi-client sequential relay implementation. CIFAR-10 is partitioned across N clients; Layer 0 weights are relayed client-to-client with full optimizer state.